CSCI 350:
Fundamentals of Operating Systems

Unix system calls

Prof. Tatyana Ryutov

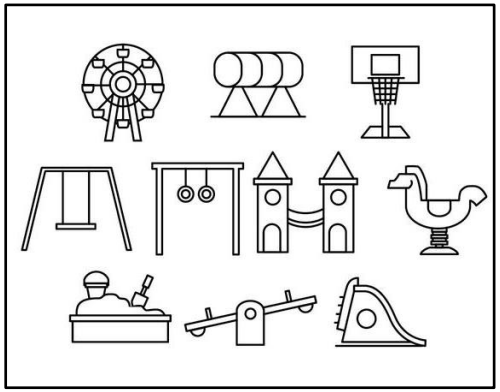
This content is protected and may not be shared, uploaded, or distributed.

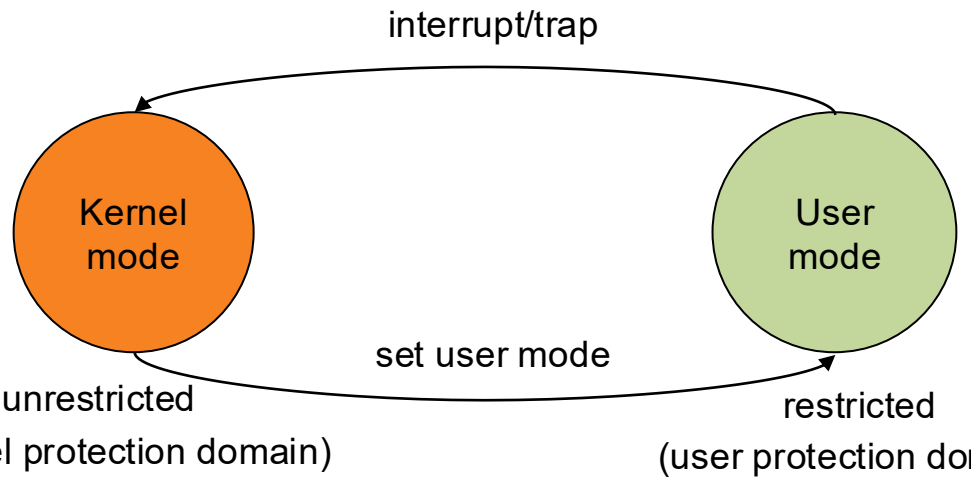
Lecture Outline

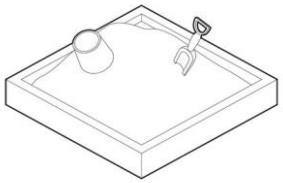
- Review
- System calls
- Kernel organization
- Unix process management
- Interrupts

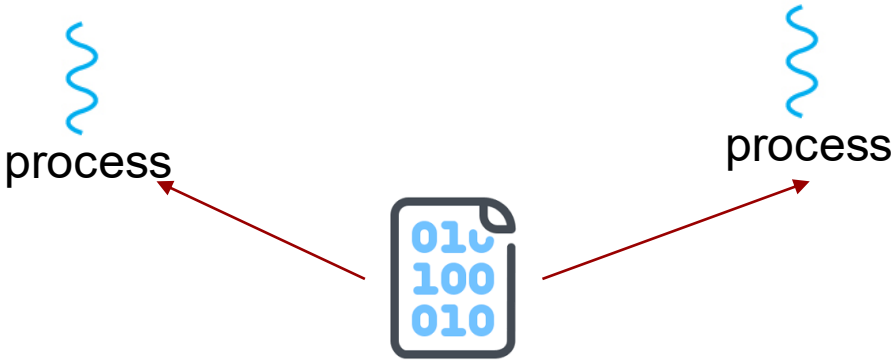
©Tatyana Ryutov

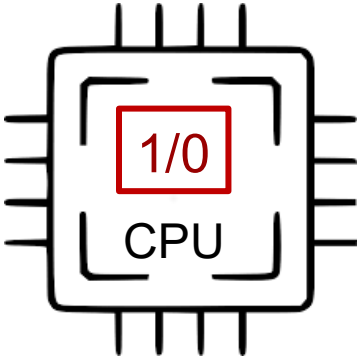
Review: Dual Mode Operation











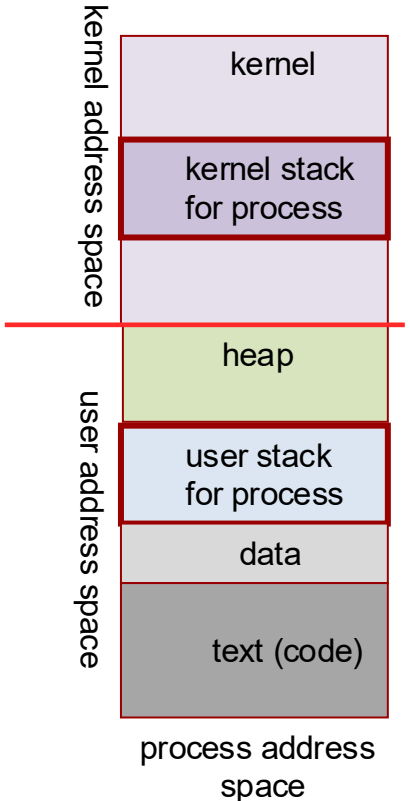
Does it make sense to have more than 2 modes?

©Tatyana Ryutov

USC Viterbi School of Engineering

Review: User and Kernel Stacks

- In most OSs each process has two stacks:
  - User space stack used when executing user code
  - Kernel space stack used when executing kernel code
- HW changes the stack pointer to point to the base of the kernel stack on event, when privilege mode switches occur
- Advantage: kernel can execute even if user stack is corrupted

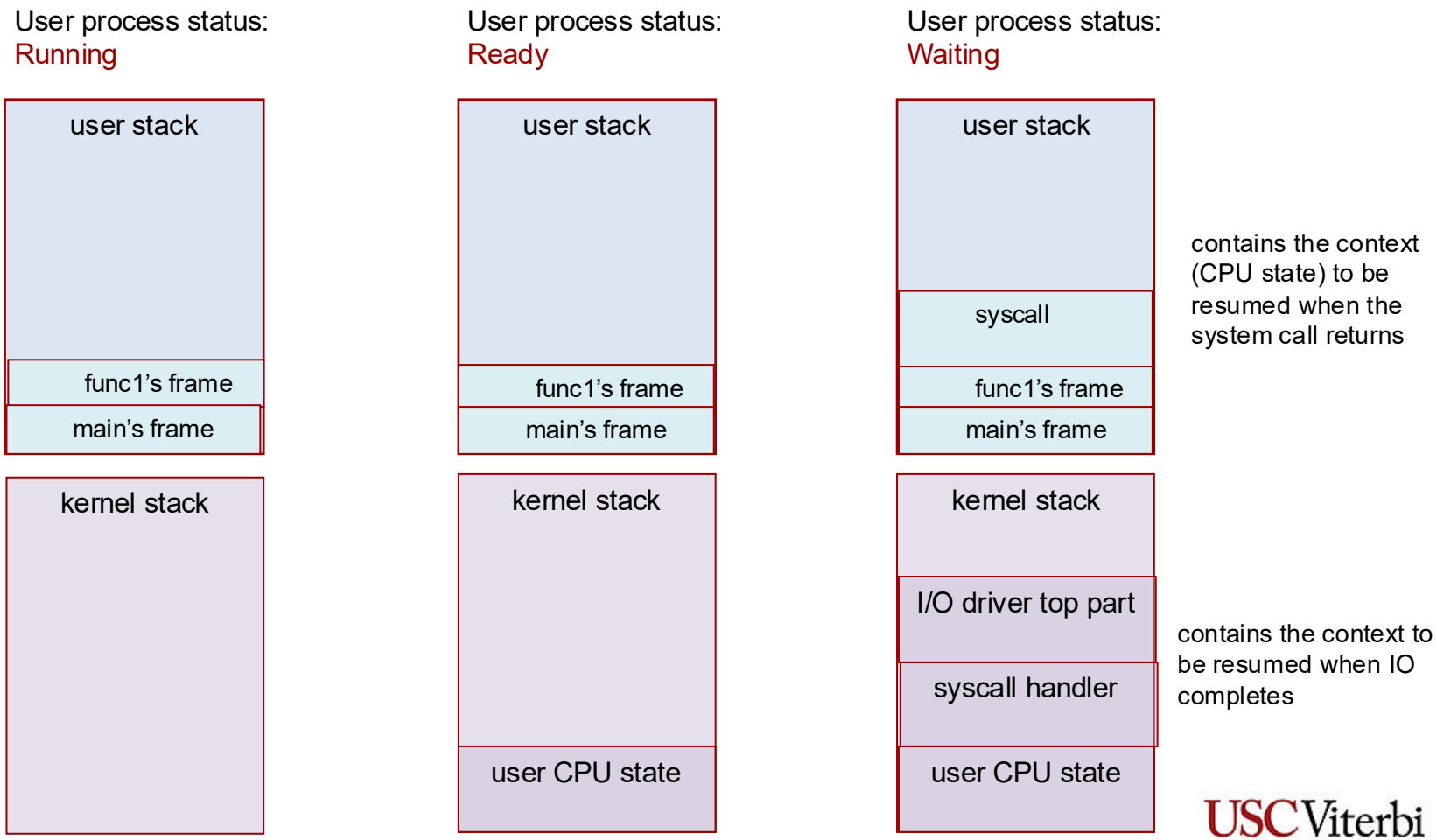


©Tatyana Ryutov

USC Viterbi
School of Engineering

When syscall occurs, what stack is used?

- On a syscall, the processor switches to the kernel stack, jumps to kernel mode, and starts executing the kernel instructions that implement the syscall



©Tatyana Ryutov

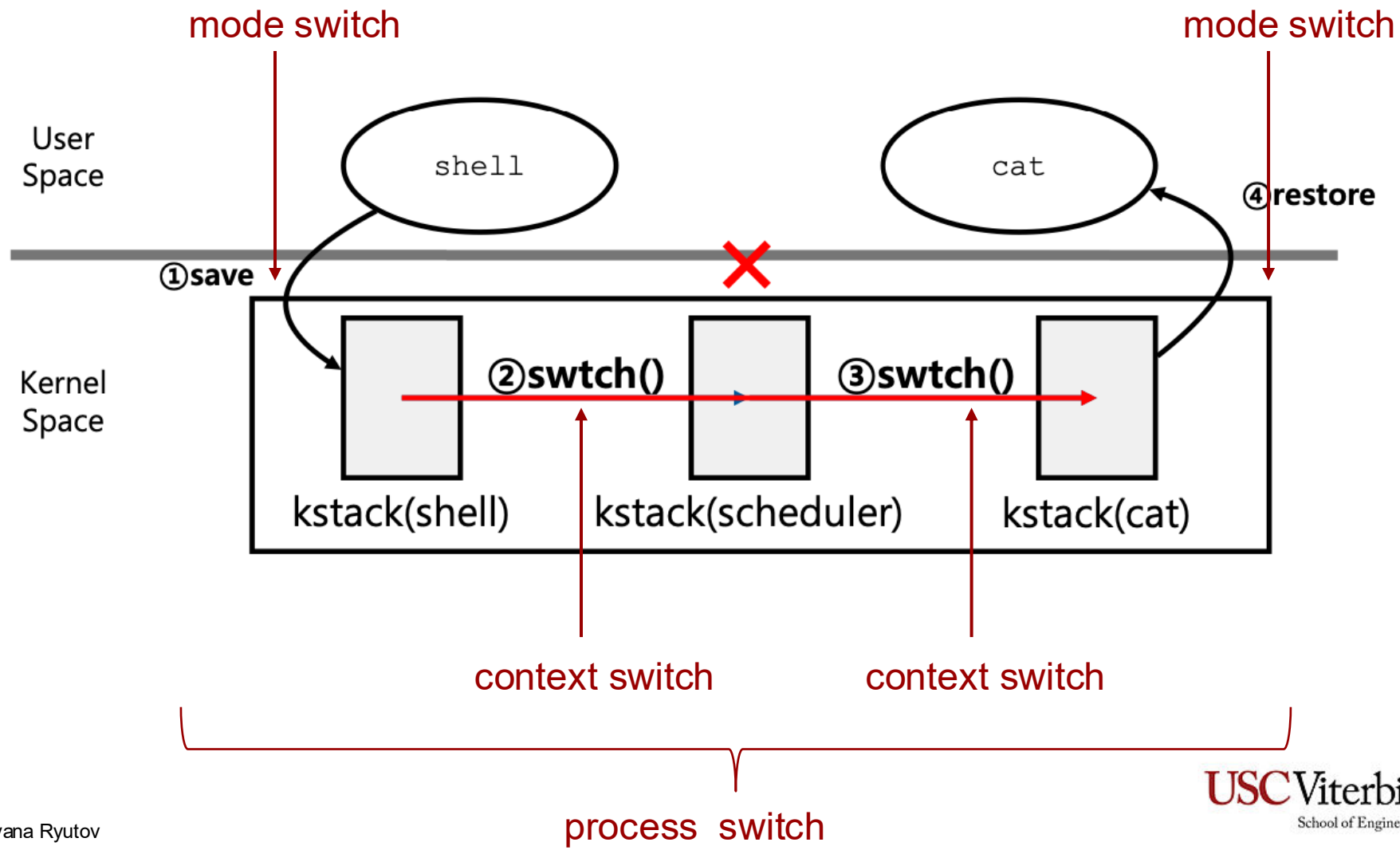
USC Viterbi School of Engineering

Review: Two Switches

- Context switch – switching CPU from one running task to another by saving the CPU state to resume the task at a later time
- Process switch - switching CPU from one process to another by saving **all of the state** of the currently executing process

©Tatyana Ryutov

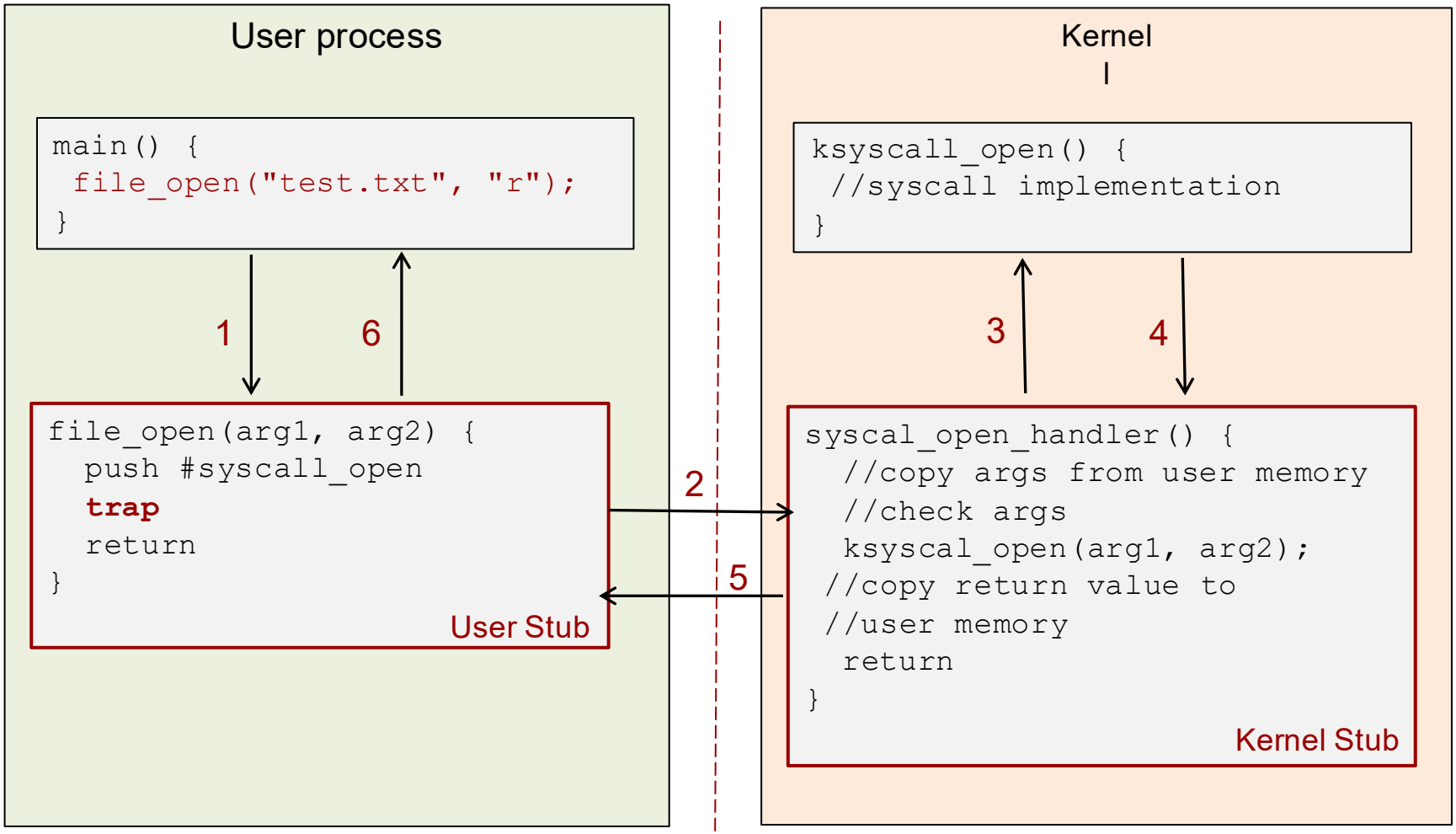
Review: x.v6 Process Switch



©Tatyana Ryutov

USC Viterbi School of Engineering

Recall: Syscall Processing Overview



User-mode execution
(code from program ELF)

Kernel-mode execution
(code from kernel binary)

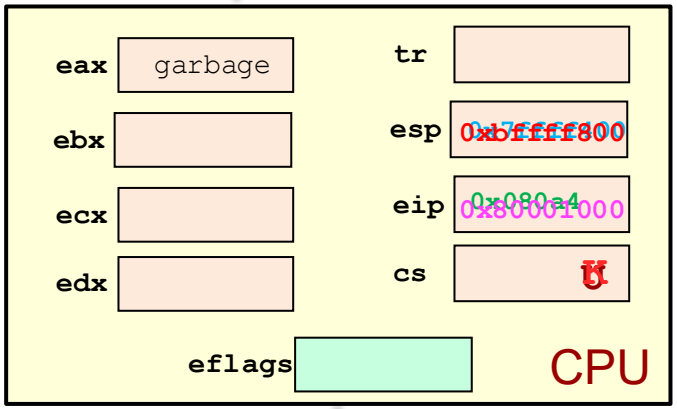
©Tatyana Ryutov

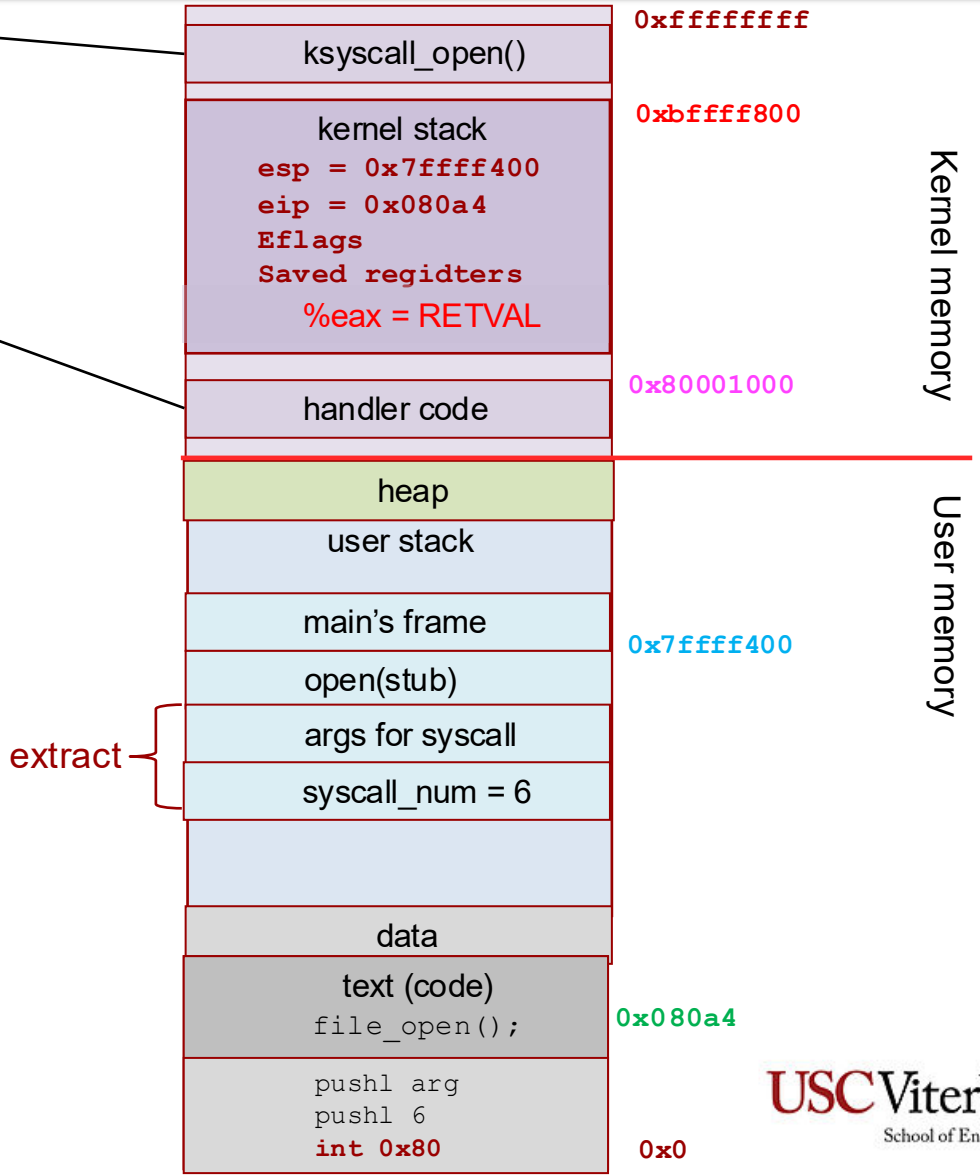
USC Viterbi School of Engineering

Syscall Execution Example

```
int ksyscall_open(…)
{
  //Extract arguments
  //Perform computation
  return value;
}
```

```
HANDLER30:
{
  pushad
  //Extract syscall num
  //If num == 0 …
  //If num = 1 …
  //...
  //If num == 6
  call ksyscall_open
  //Store rectal in %eax
   ...
   popad
  iret
}
```





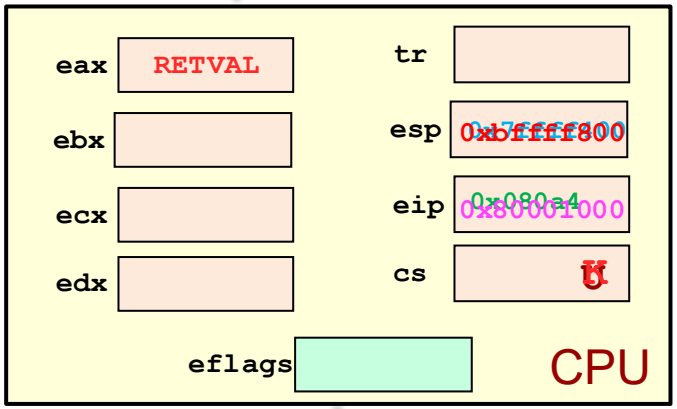
©Tatyana Ryutov

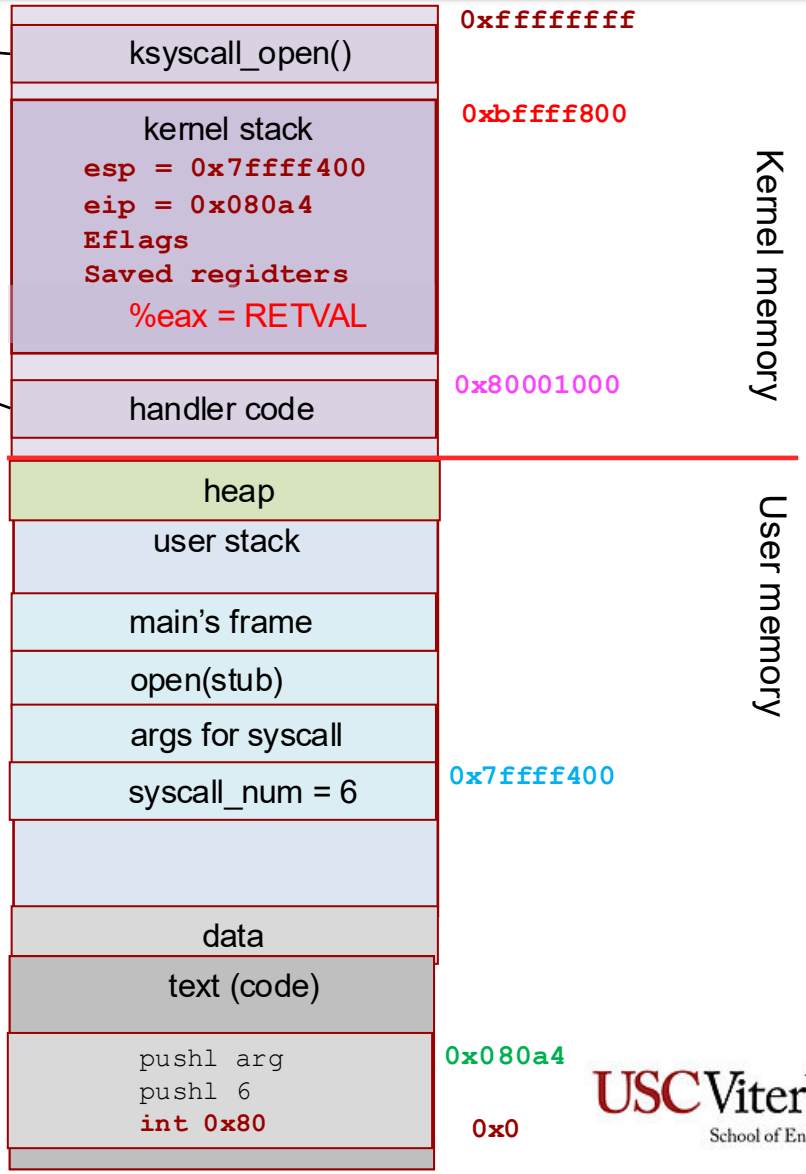
USC Viterbi School of Engineering

Syscall Execution Example

```
int ksyscall_open(…)
{
  //Extract arguments
  //Perform computation
  return value;
}
```

```
HANDLER30:
{
  pushad
  //Extract syscall num
  //If num == 0 …
  //If num = 1 …
  //...
  //If num == 6
  call ksyscall_open
  //Store rectal in %eax
   ...
   popad
  iret
}
```

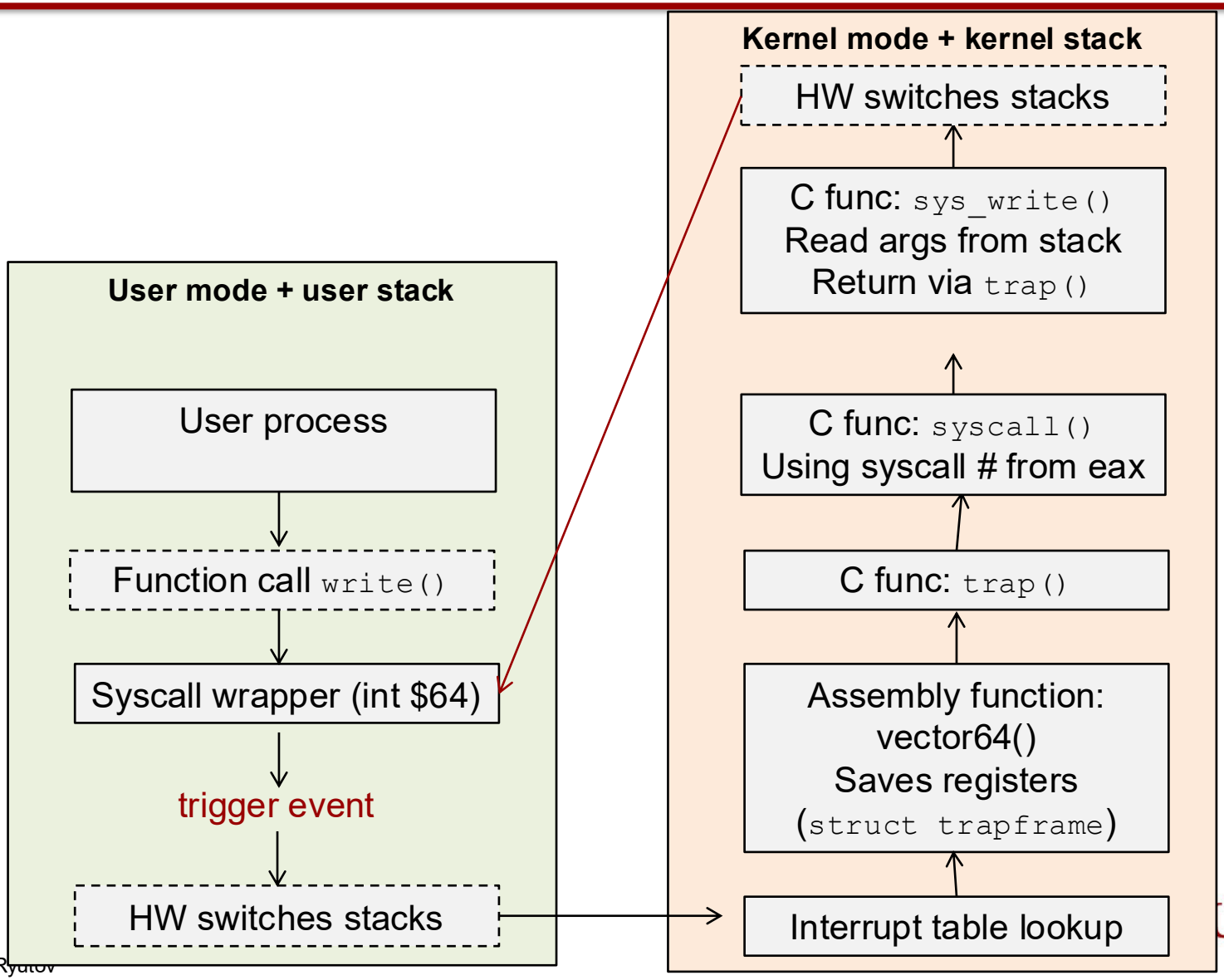




©Tatyana Ryutov

USC Viterbi School of Engineering

Syscall in xv6



©Tatyana Ryutov

xv6 write syscall: User Side

```
...
write(1, "Hello world!\n", 14);
...
```

main.c

```
...
#define SYS_write 16
...
```

syscal.h

```
...
#define T_SYSCALL 64
...
```

traps.h

(partial, after macro replacement)

```
.globl write
write:
    movl $SYS_write, %eax
    int $T_SYSCALL
    ret
```

usys.S

interrupt — trigger an event (64 in this case)

xv6 syscall calling convention: %eax = syscall number
otherwise: same as 32-bit x86 calling convention (arguments on stack)

©Tatyana Ryutov

xv6 write syscall: Kernel Side

```
void trap(struct trapframe *tf)
{
   if(tf->trapno == T_SYSCALL){
     if(myproc()->killed)
        exit();

     myproc()->tf = tf;

     syscall();

     if(myproc()->killed)
       exit();
   return;
  }
...
}
```

trap.c

`struct trapframe` — set by assembly: interrupt type, application registers, ... example: tf→eax = old value of eax

`myproc()` — represents currently running process

`syscall()` — actual implementation uses `myproc()->tf` to determine what to do

©Tatyana Ryutov

xv6 write syscall: Kernel Side

```
static int (*syscalls[])(void) = { ...
[SYS_write] sys_write,
...
};
...
void syscall(void) {
...
  num = curproc->tf->eax;
   if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
   curproc->tf->eax = syscalls[num]();  } else {
...
}
```

syscall.c

`syscalls []` array of functions — one for syscall '[number] value': syscalls[number] = value

(if system call number in range) call sys_...function from table store result in user's eax register

result assigned to eax (assembly code this returns to copies tf−>eax into %eax)

©Tatyana Ryutov

xv6 write syscall: Kernel Side

```
int sys_write(void)
{
  struct file *f;
  int n;
  char *p;

  if(argfd(0, 0, &f) < 0 || argint(2, &n) < 0 || argptr(1, &p, n) < 0)
     return -1;
  return filewrite(f, p, n);
}
```

sysfile.c

utility functions that read arguments from user's stack returns -1 on error

actual internal function that implements writing to a file

when done, trap returns to `alltraps` that restores registers from tf, then returns to user-mode

©Tatyana Ryutov

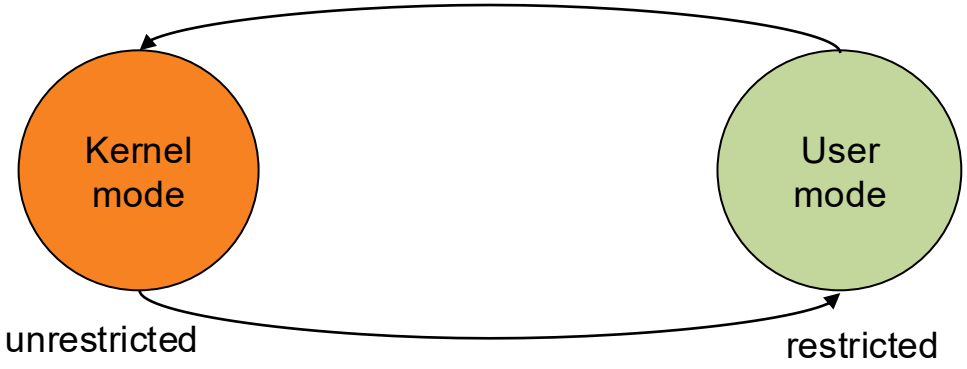
Lecture Outline

- Review
- System calls
- Kernel organization
- Unix process management
- Interrupts

©Tatyana Ryutov

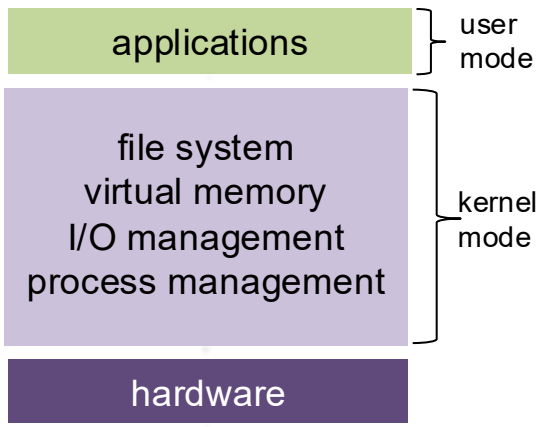
Kernel Organization

- OS must provide many services to user processes
- Key design question: where to put those services (user or kernel mode)?
- Tradeoffs: evaluate where the services should be located based on
  1. Performance
  2. Reliability/Security
  3. Flexibility/Comprehensibility
- Kernel organization approaches:
  - Monolithic
    - e.g., Windows, Linux, Mac OS
  - Layered
    - e.g., Multics, Gemsos
  - Microkernel
    - e.g., Mach, seL4
  - Virtual machine
    - e.g., Xen, VMware

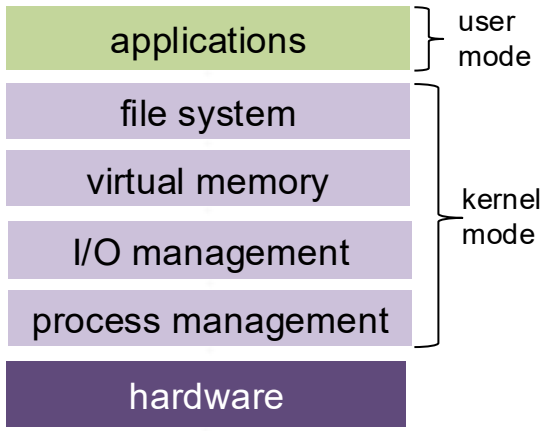


©Tatyana Ryutov

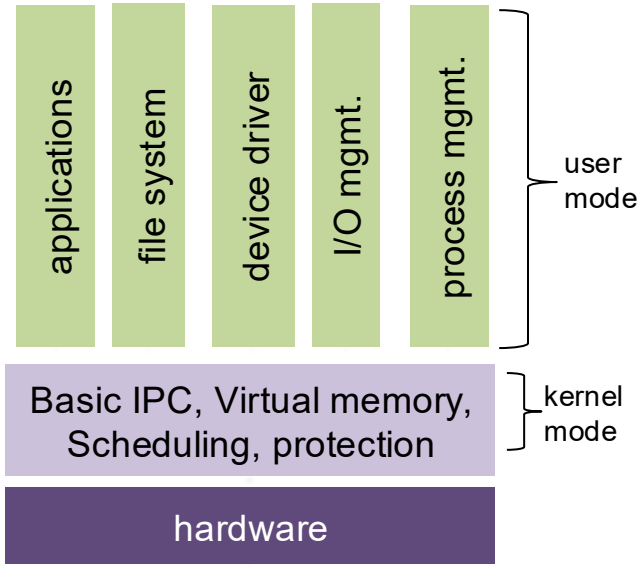
Kernel Organization Approaches



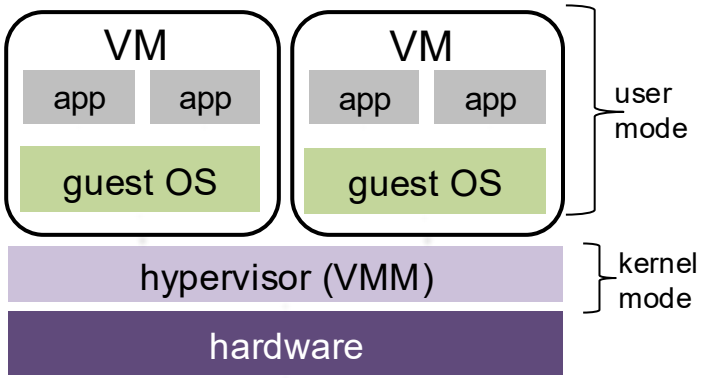
Monolithic



Layered



Microkernel

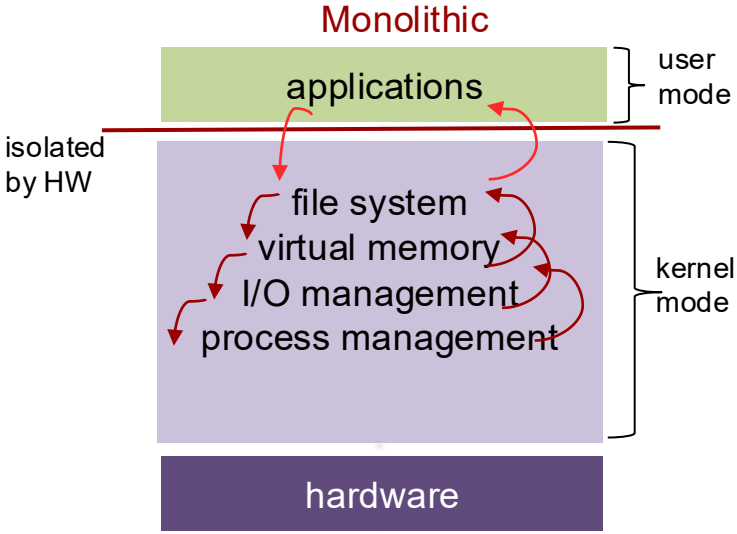


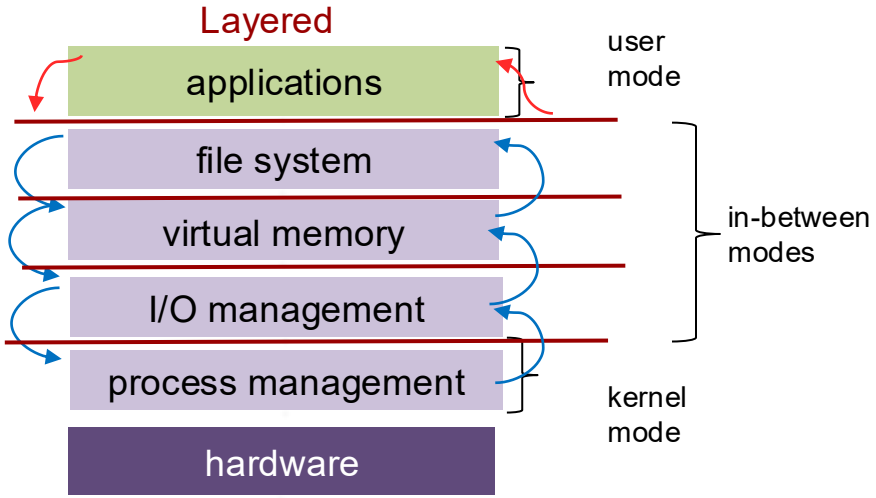
Virtual Machine

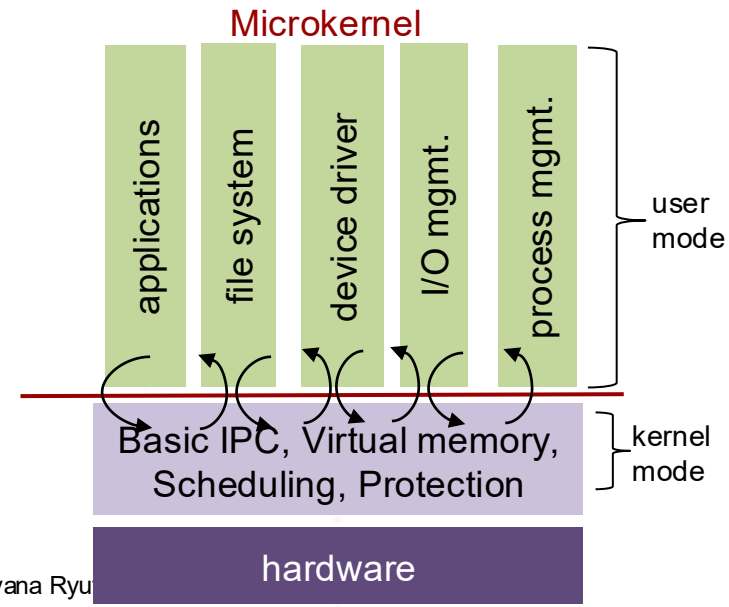
©Tatyana Ryutov

USC Viterbi School of Engineering

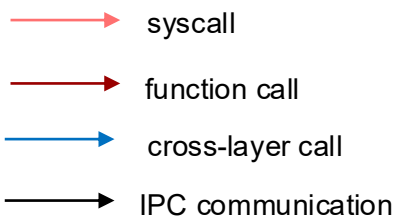
Kernel Organization Approaches and Tradeoffs







	Performance	Reliability Security	Flexibility Comprehensibility
Monolithic	best	worst	worst
Layered	bad	best	good
Microkernel	bad	good	best

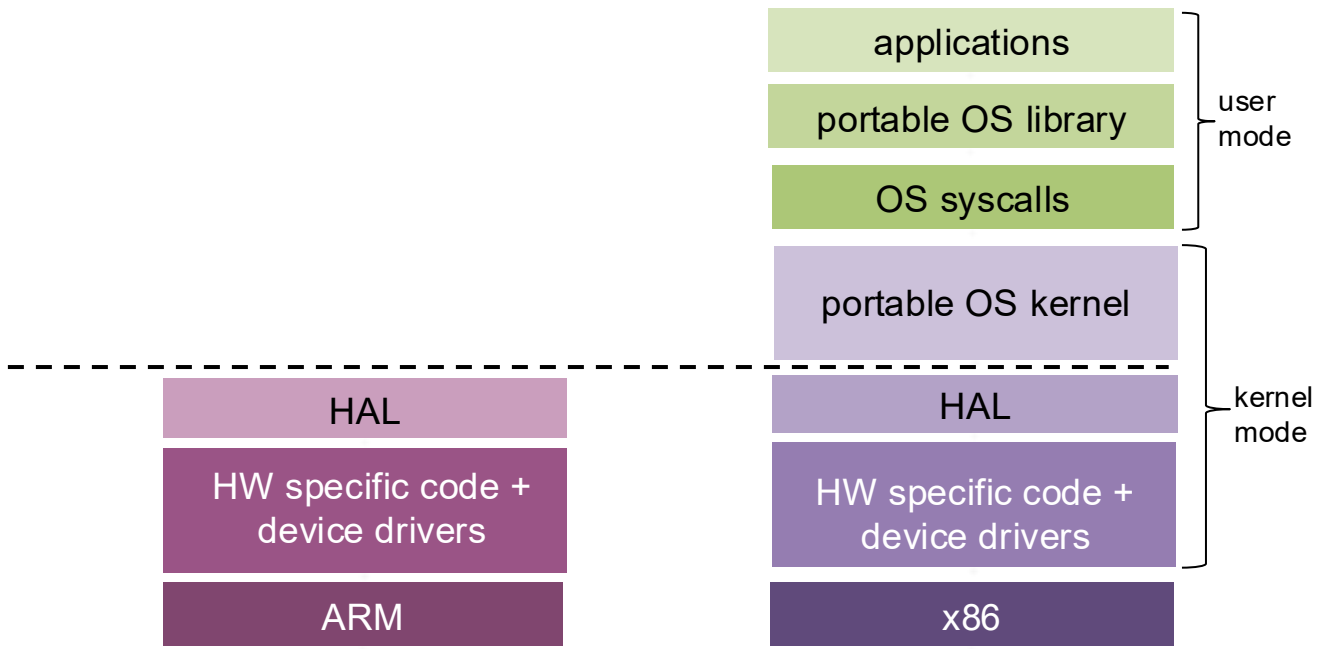


©Tatyana Ryu

USC Viterbi School of Engineering

Monolithic Kernel: Portability

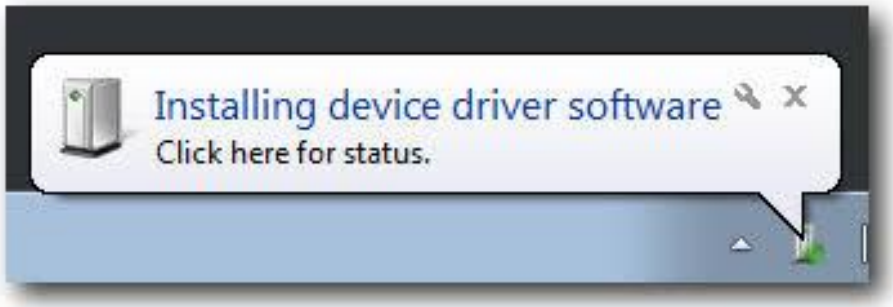
- How do we run OS on different architectures?
- General features to make OS portable and extensible
  1. Hardware Abstraction Layer (HAL)
     - Interface to machine configuration and processor-specific operations
     - Provide functions and abstract models of devices
     - To port OS to a new HW create new implementations of low-level HAL routines
  2. Dynamically loaded device drivers



©Tatyana Ryutov

Monolithic Kernel: Dynamically Installed Device Drivers

- Goal: accommodate a wide variety of physical I/O devices
- Approach:
  - Decouple the OS source code from the specifics of each device
  - Support dynamically loadable device drivers
  - The device manufacturer typically provides the driver code that supports standard interface supported by the kernel
- Tradeoffs?



©Tatyana Ryutov

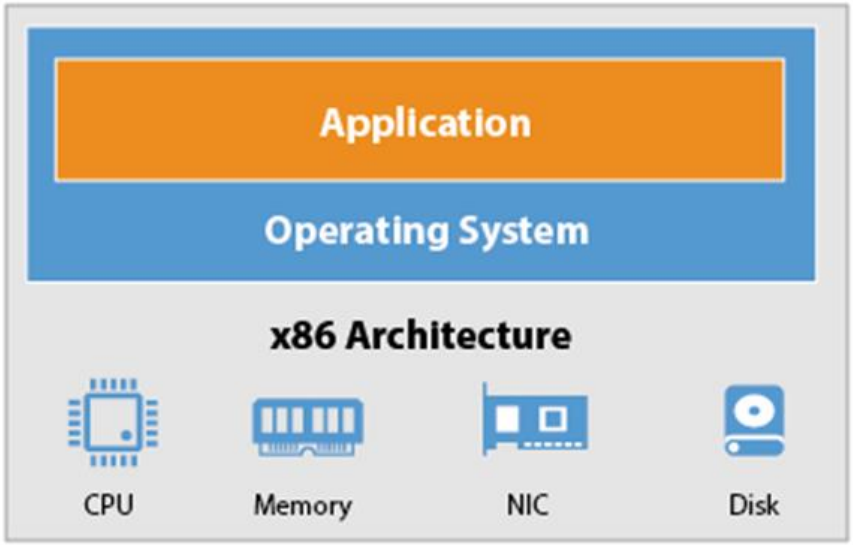
USC Viterbi School of Engineering

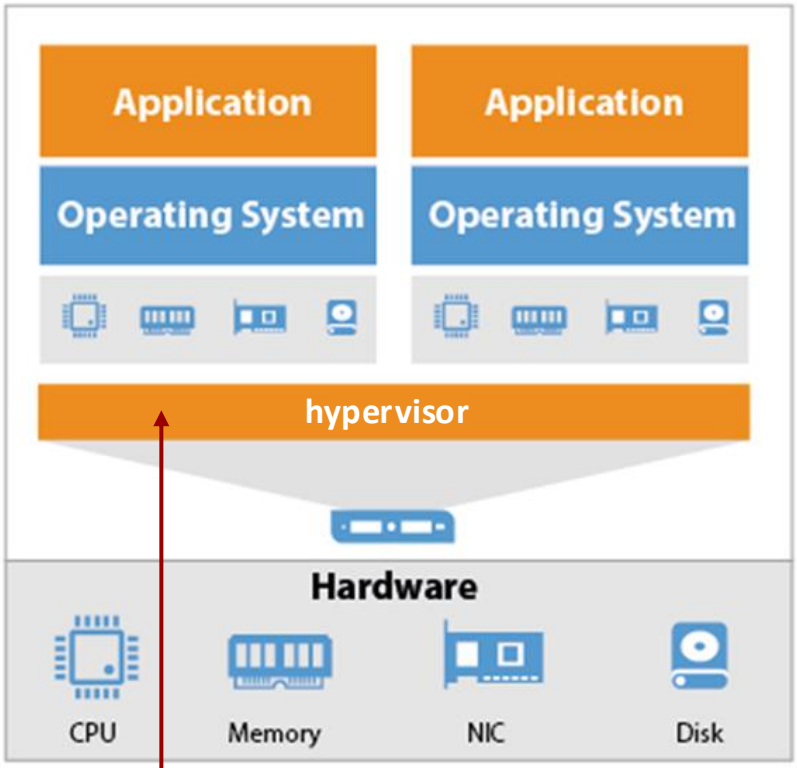
Virtualization Technology

- Virtualization uses software to create an abstraction layer over computer hardware
- Idea: abstract HW of a single computer into different execution environments
  - This creates virtual system (virtual machine, or VM)
- Components:
  - Host – underlying hardware system (or OS)
  - Virtual machine manager (VMM) or hypervisor – manages the underlying HW and provides an abstraction of one or more VMs
  - Guest – software running within a VM
- Single physical machine can run multiple OSs concurrently, each in its own VM

©Tatyana Ryutov

Physical vs. Virtual Machine



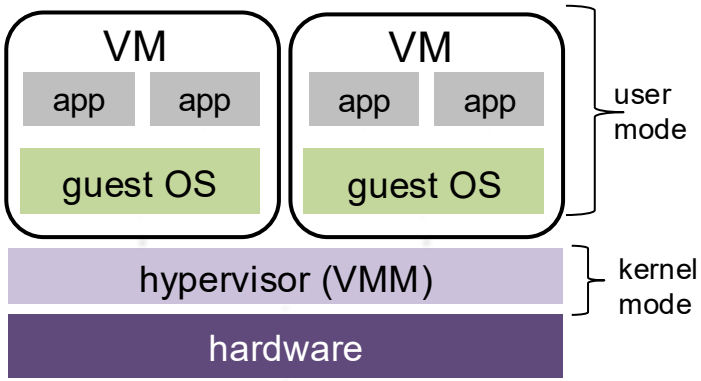


- Extra level of indirection (software abstraction):
  - Enforces isolation
  - Multiplexes physical HW across VMs

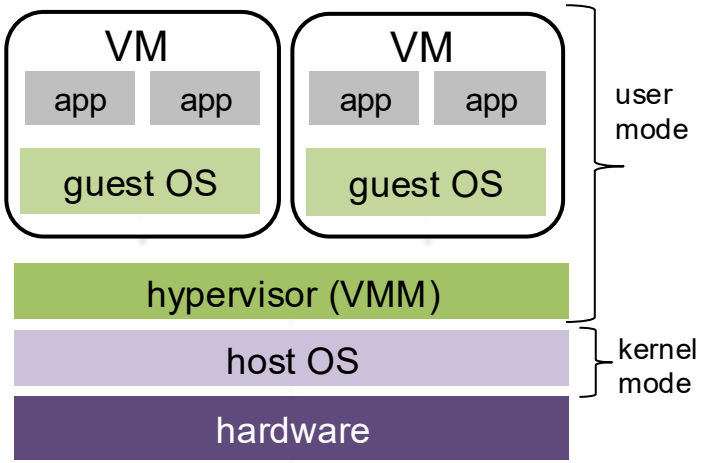
©Tatyana Ryutov

Hypervisor Types

- Native hypervisor (type 1) runs on bare HW
  - E.g., VMware vSphere with ESX/ESXi, Microsoft Hyper-V, Citrix Hypervisor (Xen Server), KVM
- Hosted hypervisor (type 2) runs on top of another OS
  - E.g., VMware Workstation, VMware Player, VirtualBox
- A combination of the native and the hosted approaches
  - Processor virtualization + hosted OS



Type 1

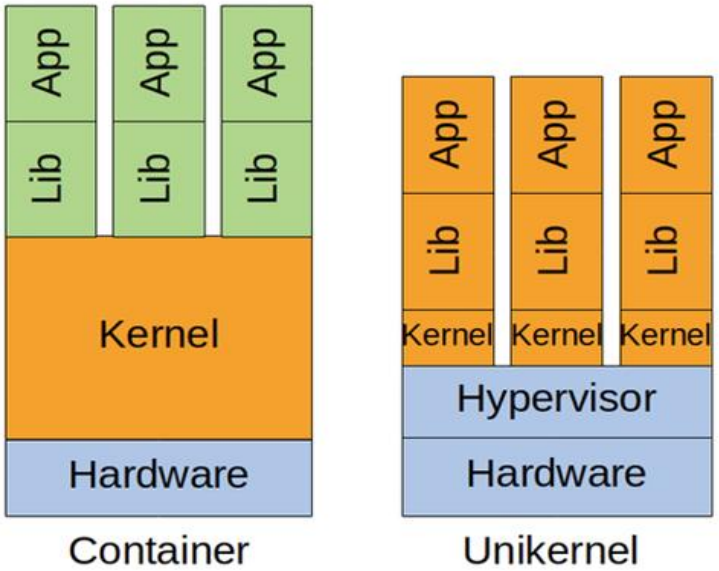


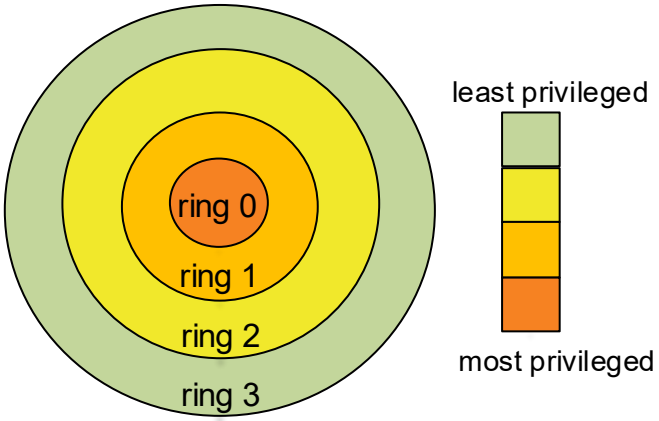
Type 2

©Tatyana Ryutov

Containers and Unikernels

- Container - lightweight VM that can choose to re-use some of the underlying OS services and libraries while overriding others
- Unikernel - special componentized OS linked directly with the application that only provides the OS components the application actually uses
- Which of these organizations provides best security/reliability?





©Tatyana Ryutov

USC Viterbi School of Engineering

Lecture Outline

- Review
- System calls
- Kernel organization
- Unix process management
- Interrupts

©Tatyana Ryutov

Process Management

- Most OSs allow one user process to create another process through system calls
- Windows approach
  - Perform process creation, environment setup, and program startup through a single system call **`createProcess(…)`**
- Unix approach
  - Process creation, environment setup, and program startup with separate system calls
  - Create process with **`fork()`** system call
  - Setup environment with possible calls to **`open(), close(), dup2()`**
  - Load program image and start execution with **`exec(…)`** system call

```
BOOL CreateProcessA(
  LPCSTR                lpApplicationName,
  LPSTR                 lpCommandLine,
  LPSECURITY_ATTRIBUTES lpProcessAttributes,
  LPSECURITY_ATTRIBUTES lpThreadAttributes,
  BOOL                  bInheritHandles,
  DWORD                 dwCreationFlags,
  LPVOID                lpEnvironment,
  LPCSTR                lpCurrentDirectory,
  LPSTARTUPINFOA        lpStartupInfo,
  LPPROCESS_INFORMATION lpProcessInformation
);
```

```
pid_t fork();
```

```
int exec( … );
```

©Tatyana Ryutov

Unix Process Management Syscalls

- **`fork`** – create a copy of the current (parent) process, and start it (child) running - no arguments!
- **`exec`** – change the program being run by the current process
- **`wait`** – wait for a process to finish
- **`exit`** – indicate that process is finished and ready to terminate

©Tatyana Ryutov

`fork`: Creating New Processes

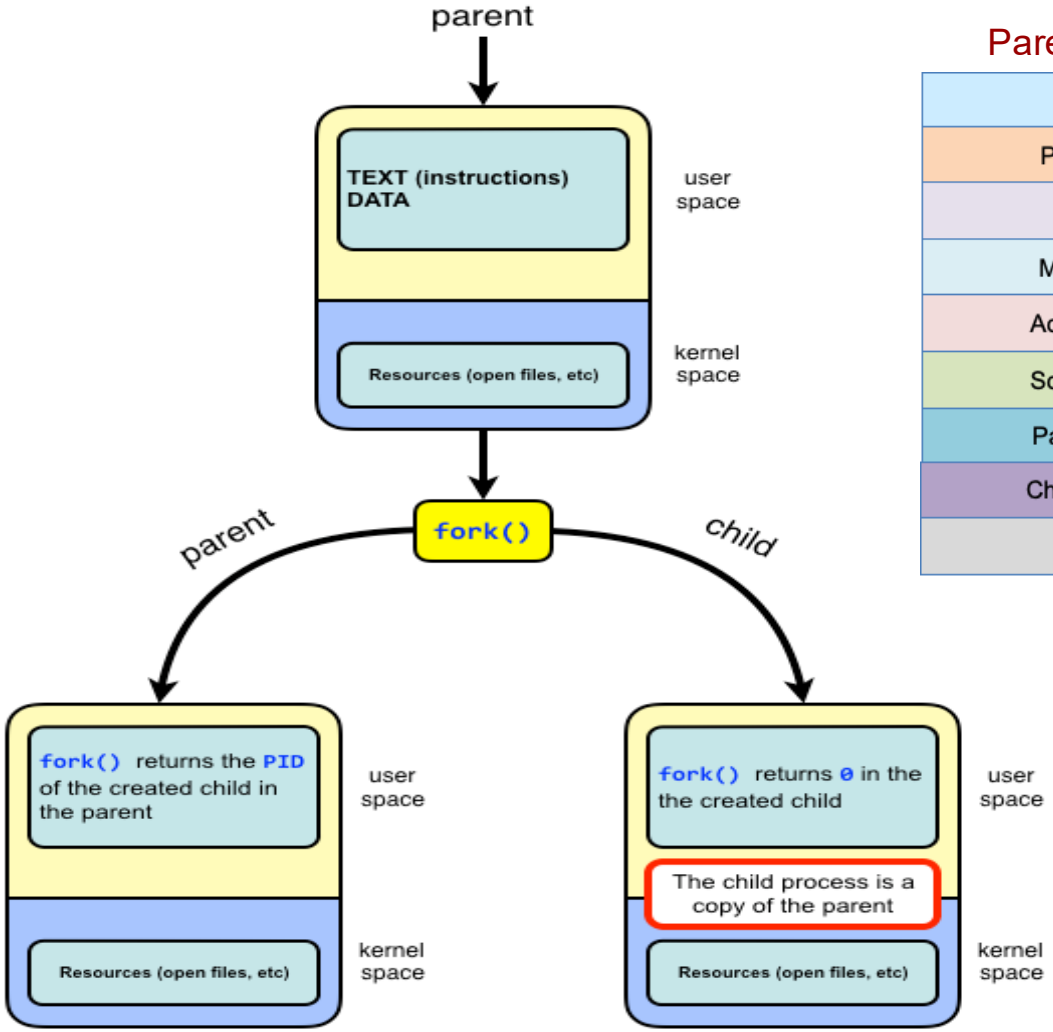
- `pid_t fork(void)`
  - Creates a new "child" process
  - Returns 0 to the child process
  - Returns child's process ID (PID) to the parent process
  - On failure, -1 is returned in the parent, no child process is created
- Child is almost identical to parent:
  - Child gets an identical (but separate) copy of the parent's virtual address space
  - Child has a different PID than the parent
- `fork` is unique (and often confusing) because it is called once but returns "twice"

```
pid_t pid = fork();
if (pid == 0) {
   printf("hello from child\n");
} else {
   printf("hello from parent\n");
}
```

©Tatyana Ryutov

System Call: `fork`

- `fork` returns twice on success
  - Parent and child execute with different memory (different address spaces)



Parent PCB	Child PCB
Process ID	Process ID
Process state	Process state
CPU state	CPU state
Memory limits	Memory limits
Accounting Info	Accounting Info
Scheduling Info	Scheduling Info
Parent process	Parent process
Child processes	Child processes
...	...

©Tatyana Ry

fork Example

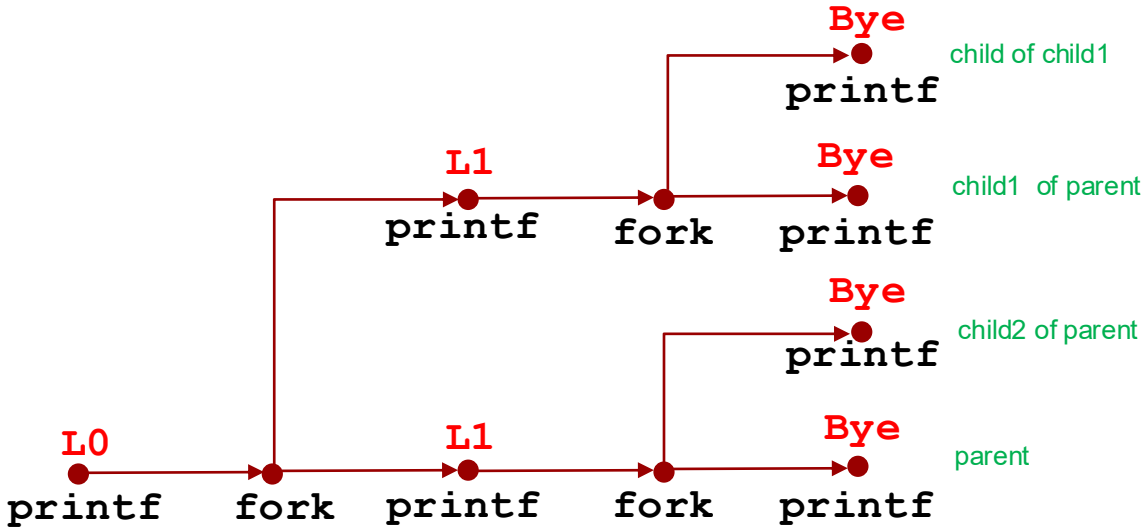
- Both processes continue/start execution after fork
  - Child starts at instruction after the call to fork (storing into pid)
  - Can't predict execution order of parent and child
- Both processes start with x = 1
  - Subsequent changes to x are independent
- Shared open files: stdout is the same in both parent and child

```
void fork1() {
    int x = 1;
    pid_t pid = fork();          ← split here
    if (pid == 0)
        printf("Child has x = %d\n", ++x);   ← child only
    else
        printf("Parent has x = %d\n", --x);  ← parent only
    printf("Bye from process %d with x = %d\n", getpid(), x);   ← both
}
```

©Tatyana Ryutov

Example: Two consecutive `fork`s

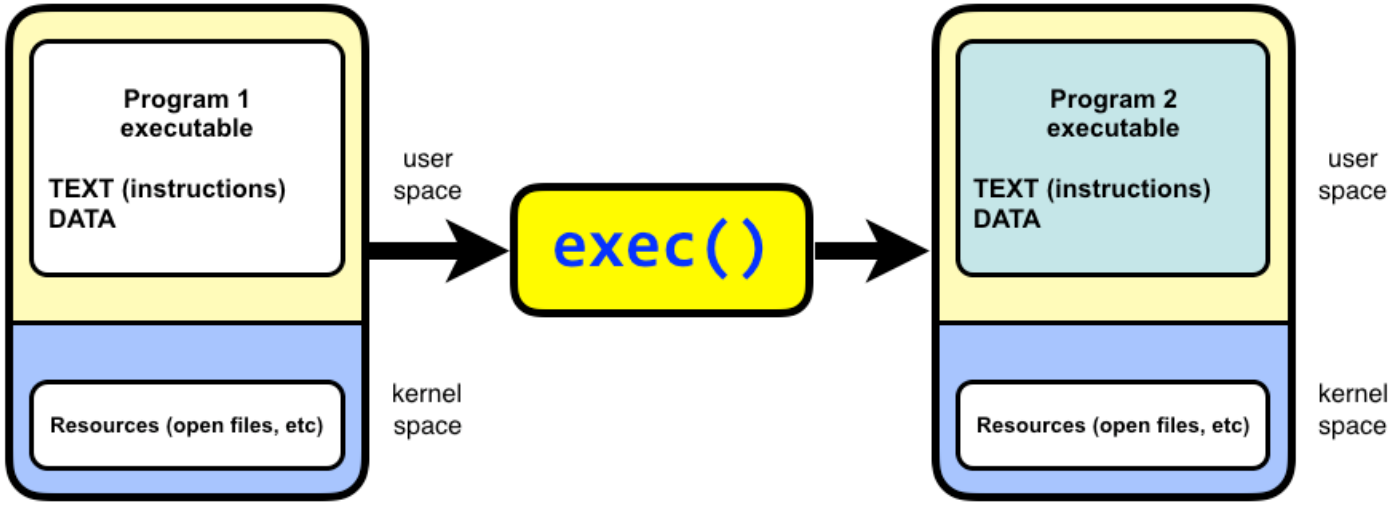
```
void fork2() {
    printf("L0\n");
    fork();
    printf("L1\n");
    fork();
    printf("Bye\n");
}
```



©Tatyana Ryutov

System Calls: `exec`

- **`exec`** replaces the data and program (text section) executed by a process
- Child process typically calls **`exec`** once it has returned from **`fork`** and configured the execution environment for the new process
- All open file descriptors remain open after calling **`exec`** unless explicitly set to **`close-on-exec`**



©Tatyana Ryutov

USC Viterbi School of Engineering

ecxec Family Functions

```
#include <unistd.h>
```

```
int execl(const char* path, const char* arg, …)
int execlp(const char* file, const char* arg, …)
int execle(const char* path, const char* arg, …, char* const envp[])
int execv(const char* path, const char* argv[])
int execvp(const char* file, const char* argv[])
int execvpe(const char* file, const char* argv[], char *const envp[])
```

- **path** - full path name of the file which is to be executed
- **arg** - the argument passed
- **const char* arg** - a list of pointers to null terminated strings
- **envp** - an array which contains pointers that point to the environment variables
- **file** - the path name which will identify the path of new process image file

©Tatyana Ryutov

exec example

example.c

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    printf("PID of example.c = %d\n", getpid());
    char *args[] = {"Hello", "C", "Programming", NULL};
    execv("./hello", args);
    printf("Back to example.c");
    return 0;
}
```

hello.c

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    printf("We are in Hello.c\n");
    printf("PID of hello.c = %d\n", getpid());
    return 0;
}
```

ubuntu@ubuntu: ~/Documents

```
ubuntu@ubuntu:~/Documents$ gcc -o example example.c
ubuntu@ubuntu:~/Documents$ gcc -o hello hello.c
ubuntu@ubuntu:~/Documents$ ./example
PID of example.c = 4733
We are in Hello.c
PID of hello.c = 4733
ubuntu@ubuntu:~/Documents$
```

output

©Tatyana Ryutov

fork-exec

- **fork-exec** model:
  - `fork()` creates a copy of the current process
  - `exec*()` replaces the current process code with the code for a different program
- The return values of `fork` and `exec*` should be checked for errors

```
// Example arguments: path="/usr/bin/ls",
// argv[0]="/usr/bin/ls", argv[1]="-ahl", argv[2]=NULL

void fork_exec(char *path, char *argv[]) {
   pid_t pid = fork();
   if (pid != 0) {
      printf("Parent: created a child %d\n", pid);
   } else {
      printf("Child: about to exec a new program\n");
      execv(path, argv);
   }
   printf("This line printed by parent only!\n");
}
```

©Tatyana Ryutov

* Represents various flavors of exec

USC Viterbi School of Engineering

fork-exec example

example.c

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main(int argc, char *argv[])
{
    printf("PID of example.c = %d\n", getpid());
    pid_t p;
    p = fork();
    if(p==-1)
    {
        printf("There is an error while calling fork()");
    }
    if(p==0)
    {
    printf("We are in the child process\n");
    printf("Calling hello.c from child process\n");
    char *args[] = {"Hello", "C", "Programming", NULL};
    execv("./hello", args);
    }
    else
    {
        printf("We are in the parent process");
    }
    return 0;
}
```

hello.c

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    printf("We are in Hello.c\n");
    printf("PID of hello.c = %d\n", getpid());
    return 0;
}
```

ubuntu@ubuntu: ~/Documents

```
ubuntu@ubuntu:~/Documents$ gcc -o example example.c
ubuntu@ubuntu:~/Documents$ gcc -o hello hello.c
ubuntu@ubuntu:~/Documents$ ./example
PID of example.c = 4790
We are in Parent Process
ubuntu@ubuntu:~/Documents$ We are in Child Process
Calling hello.c from child process
We are in Hello.c
PID of hello.c = 4791
```

output

©Tatyana Ryutov

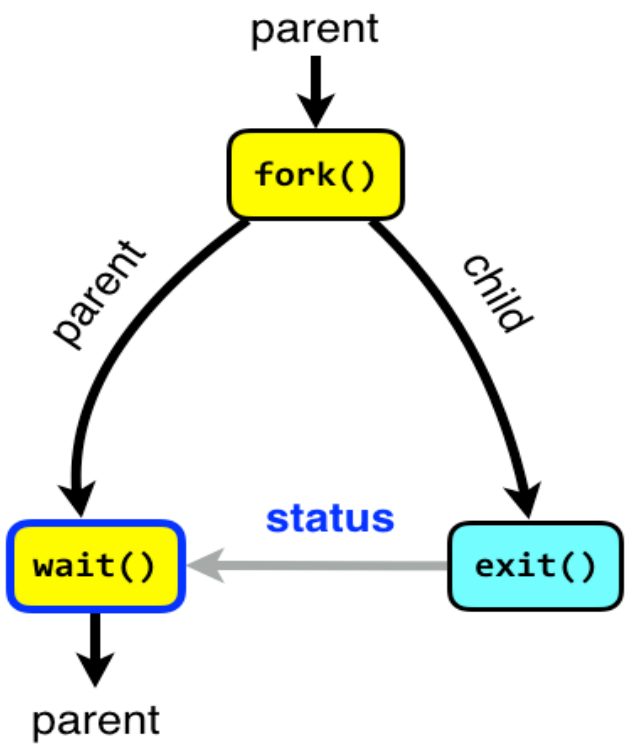
`fork-exec` Summary

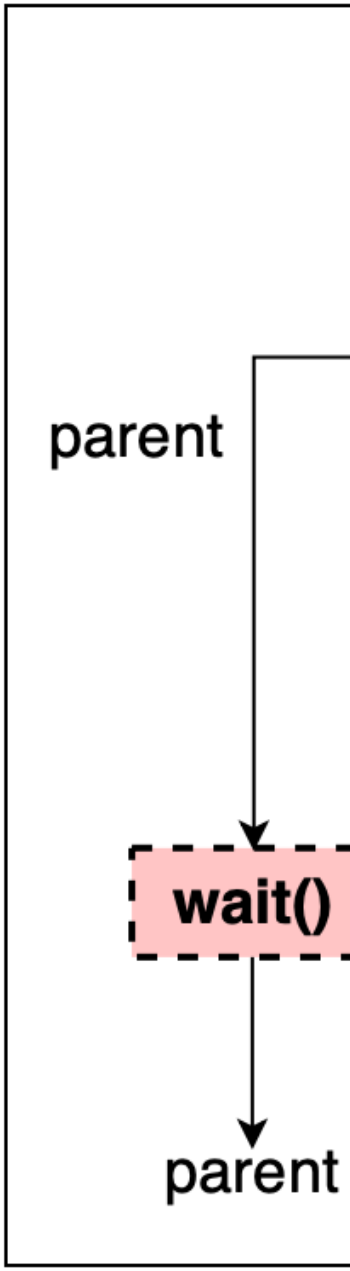
- `fork` causes the kernel to
  - Create an initialize the PCB entry for a new process
  - Create a new address space and initialize it with a complete copy of the parent's address space
  - Inherit the context of the parent (open files, etc.)
  - Inform the scheduler that the new process is ready to run
- `exec` causes the kernel to
  - Load a new program image into the current address space
  - Copy arguments into the address space
  - Initialize the CPU context (PC + stack) to the "start" entry point of the program image
  - `exec` does not create a new process!

©Tatyana Ryutov

System Call: `wait`

- **`wait`** blocks the caller (parent process) until one of its child terminates
  - If the parent does not have any child processes, **`wait`** returns immed blocking the parent
  - Parent does not wait if it forks a background process
- Using **`wait`** the parent can obtain the exit status of the term





©Tatyana Ryutov

`wait`: Synchronizing with Children

- `pid_t wait(int *child_status)`
  - Suspends current process (i.e. the parent) until one of its children terminates
  - Return value is the PID of the child process that terminated
  - If `child_status != NULL`, then `*child_status` indicates why the child terminated
- **Child status** information reported by wait includes:
  - normal/abnormal termination
  - termination cause
  - exit status
  - examined using macros `WIFEXITED(), WEXITSTATUS()`, etc.
- **Note:** If parent process has multiple children, `wait` returns when **any** of the children terminates
  - To wait on specific child use `waitpid()` function which is parametrized with a child PID

```
void fork_wait() {
   int child_status;

   if (fork() == 0) {
      printf("HC: hello from child\n");
      exit(0);
   } else {
      printf("HP: hello from parent\n");
      wait(&child_status);
      printf("CT: child has terminated\n");
   }
   printf("Bye\n");
}
```

©Tatyana Ryutov

System Call: `exit`

- **`void exit (int status)`** – a user process calls `exit` to indicate that it is finished and ready to terminate
- Exit status is returned to OS and `SIGCHLD` signal is sent to the parent process
- The exit status can then be retrieved by the parent process via the `wait` system call

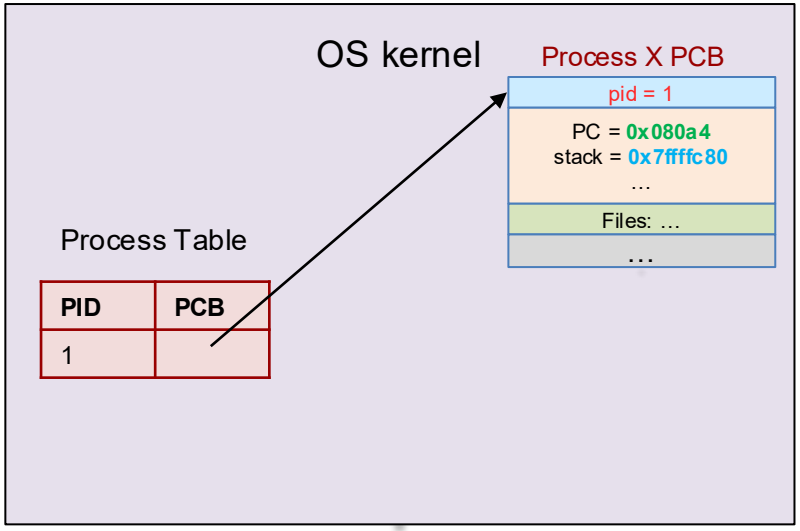
©Tatyana Ryutov

Example: `fork`

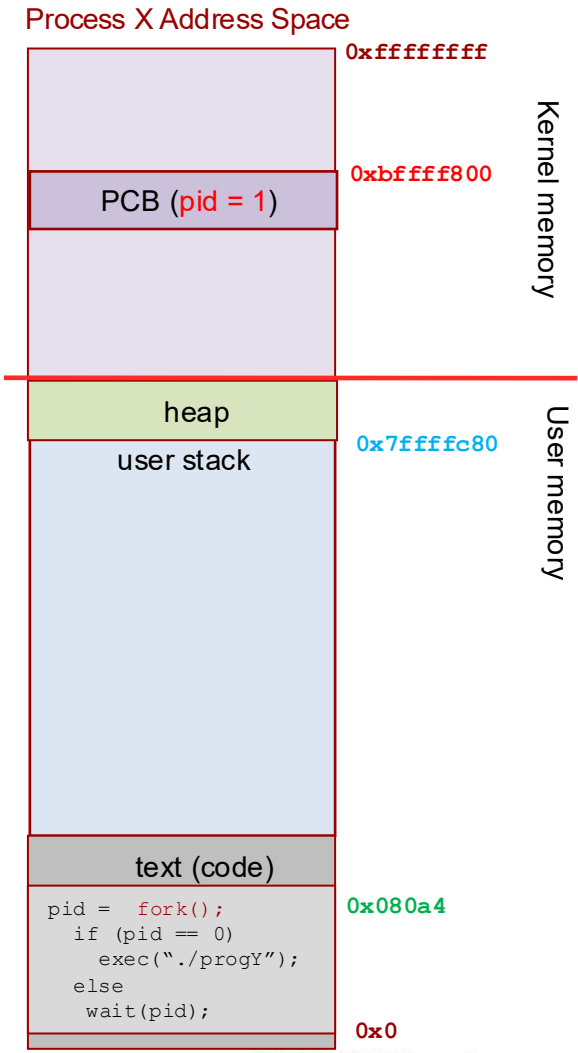
- User process X (pid = 1) is running and wants to start another

Parent process X (/.progX)

```
pid = fork();
  if (pid == 0)
    exec("./progY");
  else
   wait(pid);
```



Abstract view



©Tatyana Ryutov

USC Viterbi School of Engineering

Example: `fork`

- `fork` copies the state of the current (parent) process to a new child process

Parent process X (/.progX)

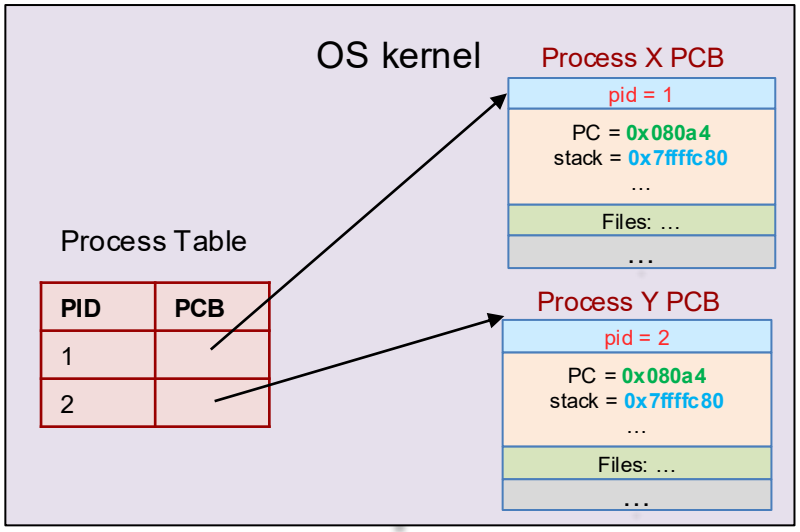
```
pid =  fork();
  if (pid == 0)
    exec("./progY");
  else
   wait(pid);
```

pid =2

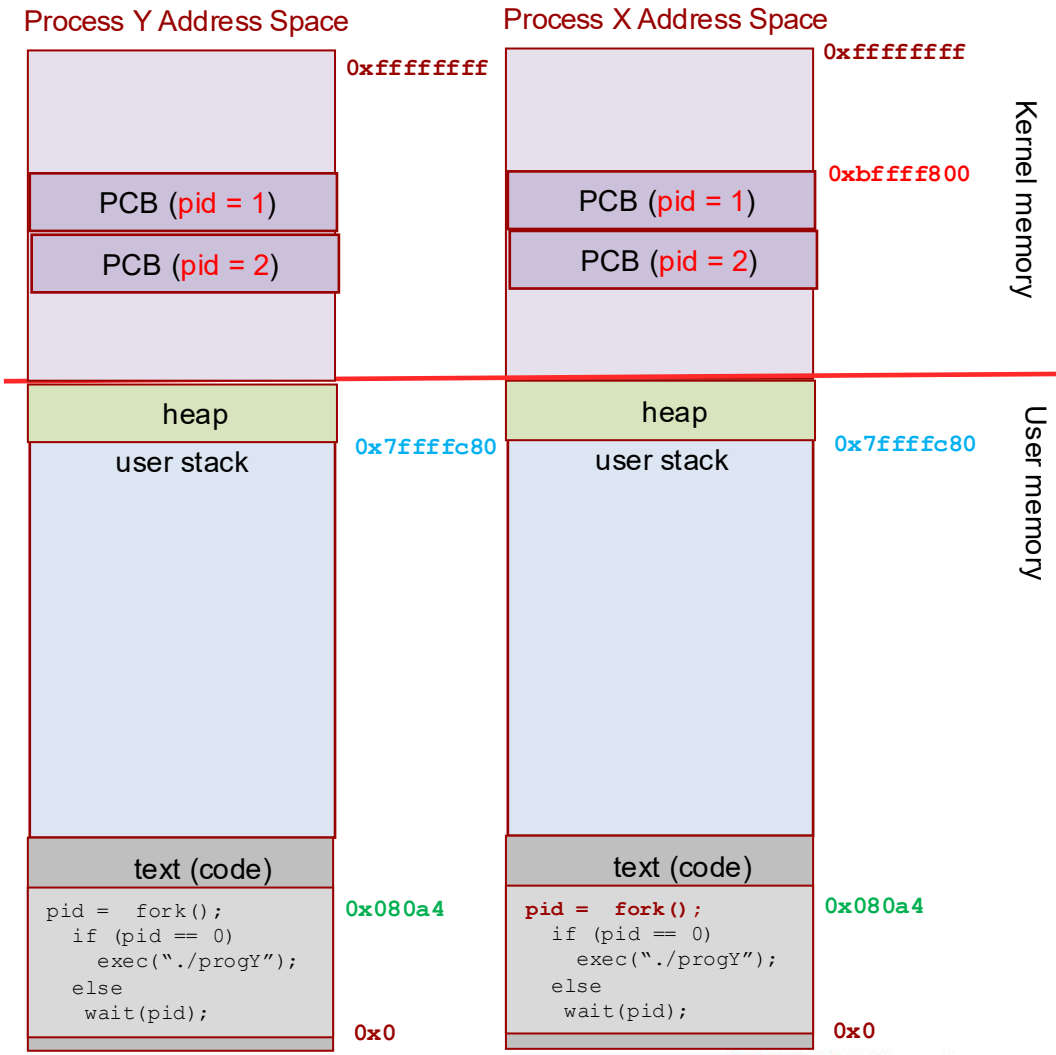
Child process Y (/.progY)

```
pid =  fork();
  if (pid == 0)
    exec("./progY");
  else
   wait(pid);
```

pid =0



Abstract view



©Tatyana Ryutov

Example: `exec`

- `exec` loads a new program (e.g., ./progY) with its own code, sets up its global data, stack, heap, etc.

Parent process X (/.progX)

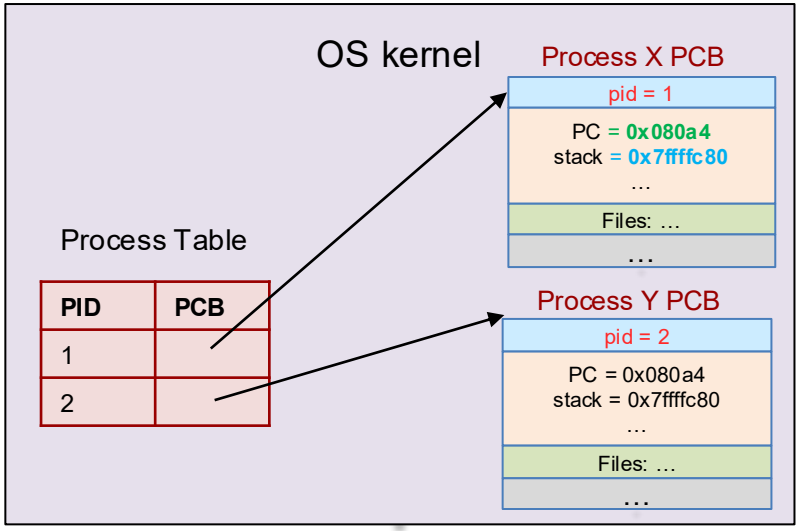
```
pid =  fork();
  if (pid == 0)
    exec("./progY");
  else
   wait(pid);
```

pid =2

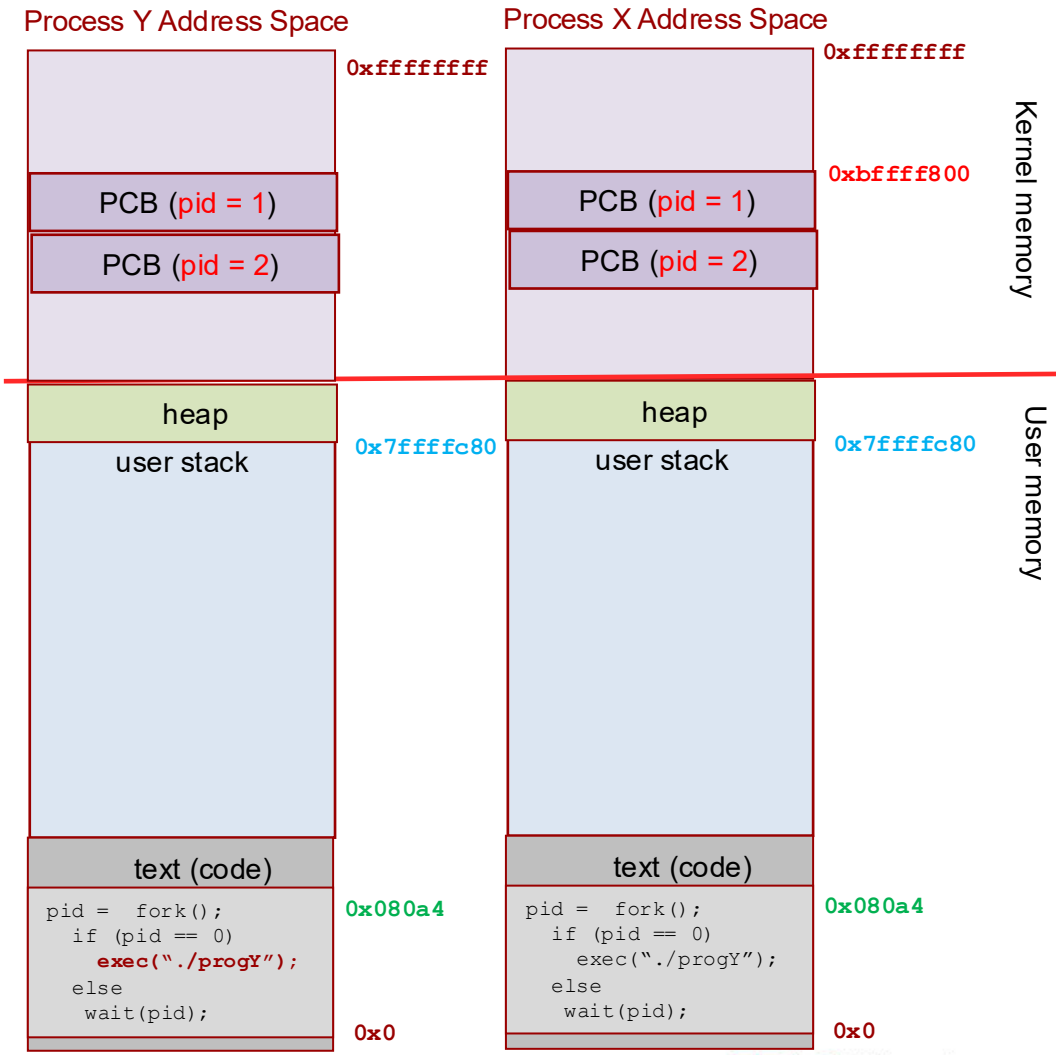
Child process Y (/.progY)

```
pid =  fork();
  if (pid == 0)
    exec("./progY");
  else
   wait(pid);
```

pid =0



Abstract view



©Tatyana Ryutov

USC Viterbi School of Engineering

Example: `exec`

- The effect of a call to `exec` is a separate program image is loaded and begins execution
- If the parent calls `wait` then it will block until the child process calls exit

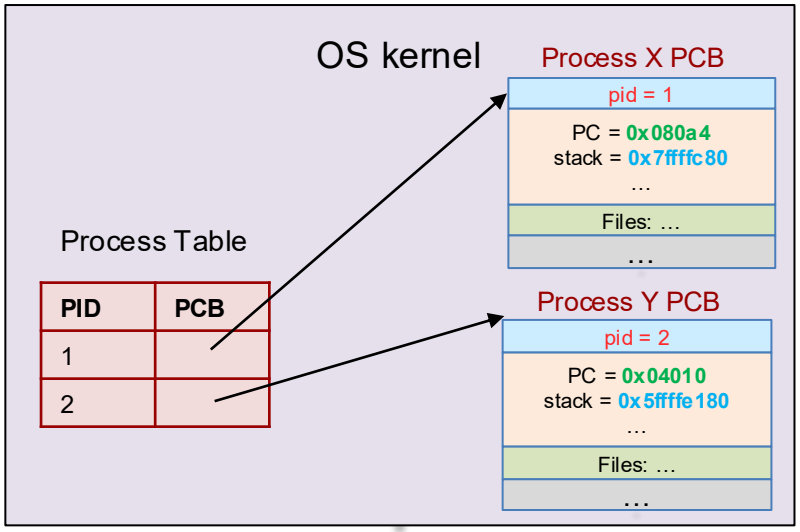
Parent process X (/.progX)

```
pid =  fork();
  if (pid == 0)
    exec("./progY");
  else
   wait(pid);
```

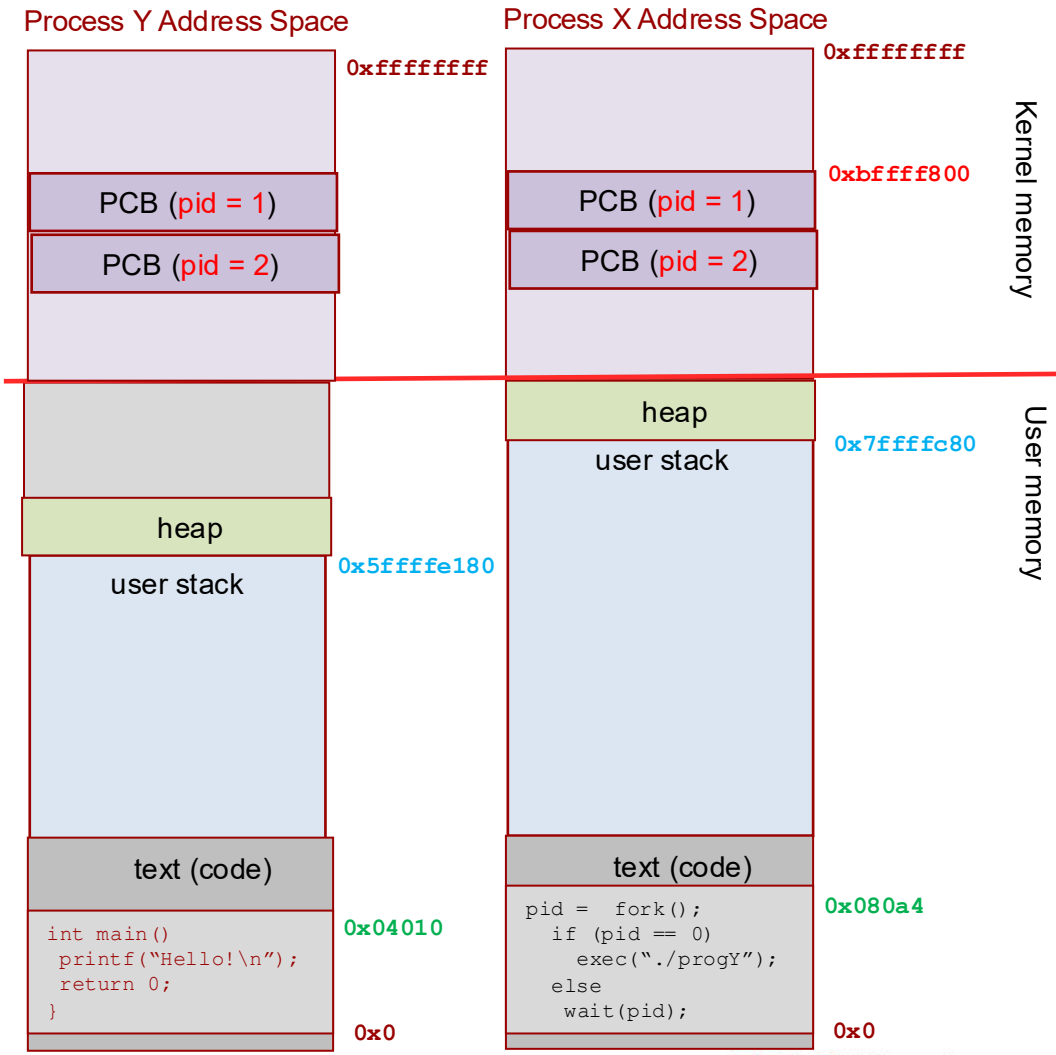
pid =2

Child process Y (/.progY)

```
int main()
 printf("Hello!\n");
 return 0;
}
```



Abstract view



©Tatyana Ryutov

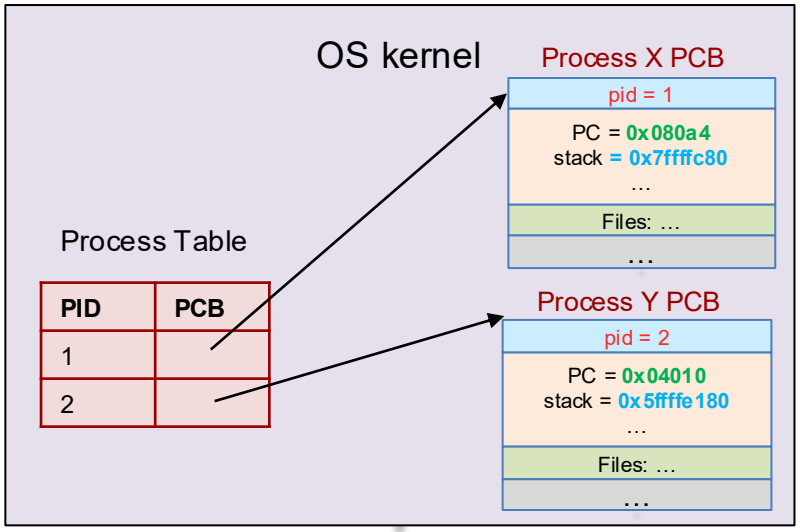
Example: `exit`

- `exit` deletes the current process and its resources
- PCB remains until explicitly deallocated by another entity

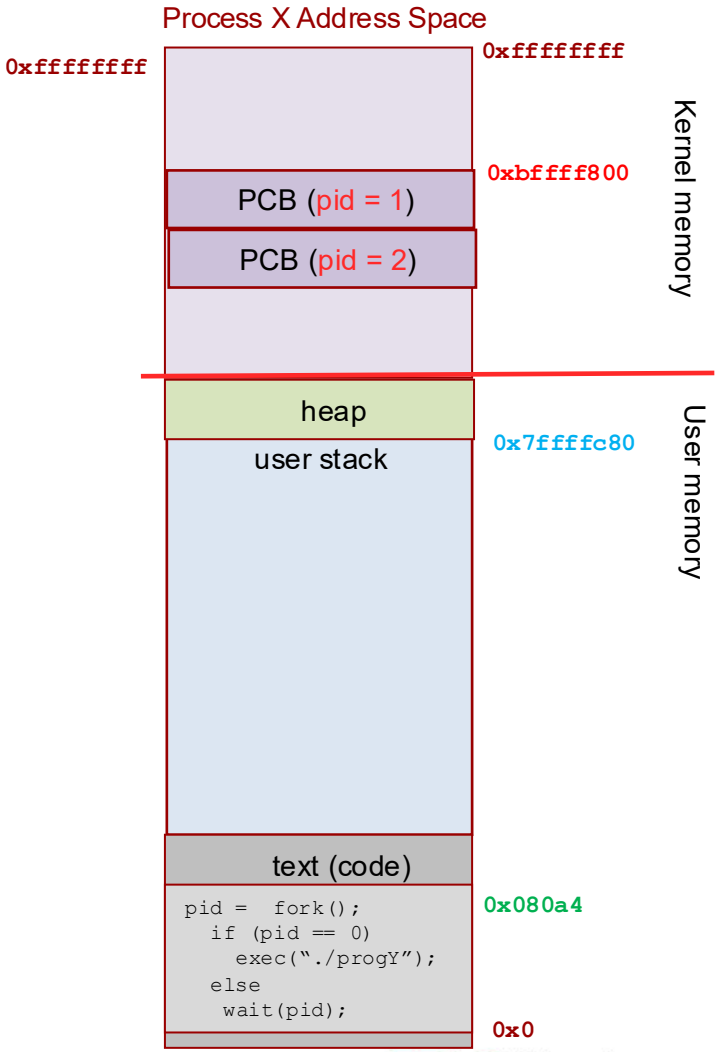
Parent process X (/.progX)

```
pid =  fork();
  if (pid == 0)
    exec("./progY");
  else
   wait(pid);
```

pid =2



Abstract view



©Tatyana Ryutov

USC Viterbi School of Engineering

Zombie (or Defunct) Process

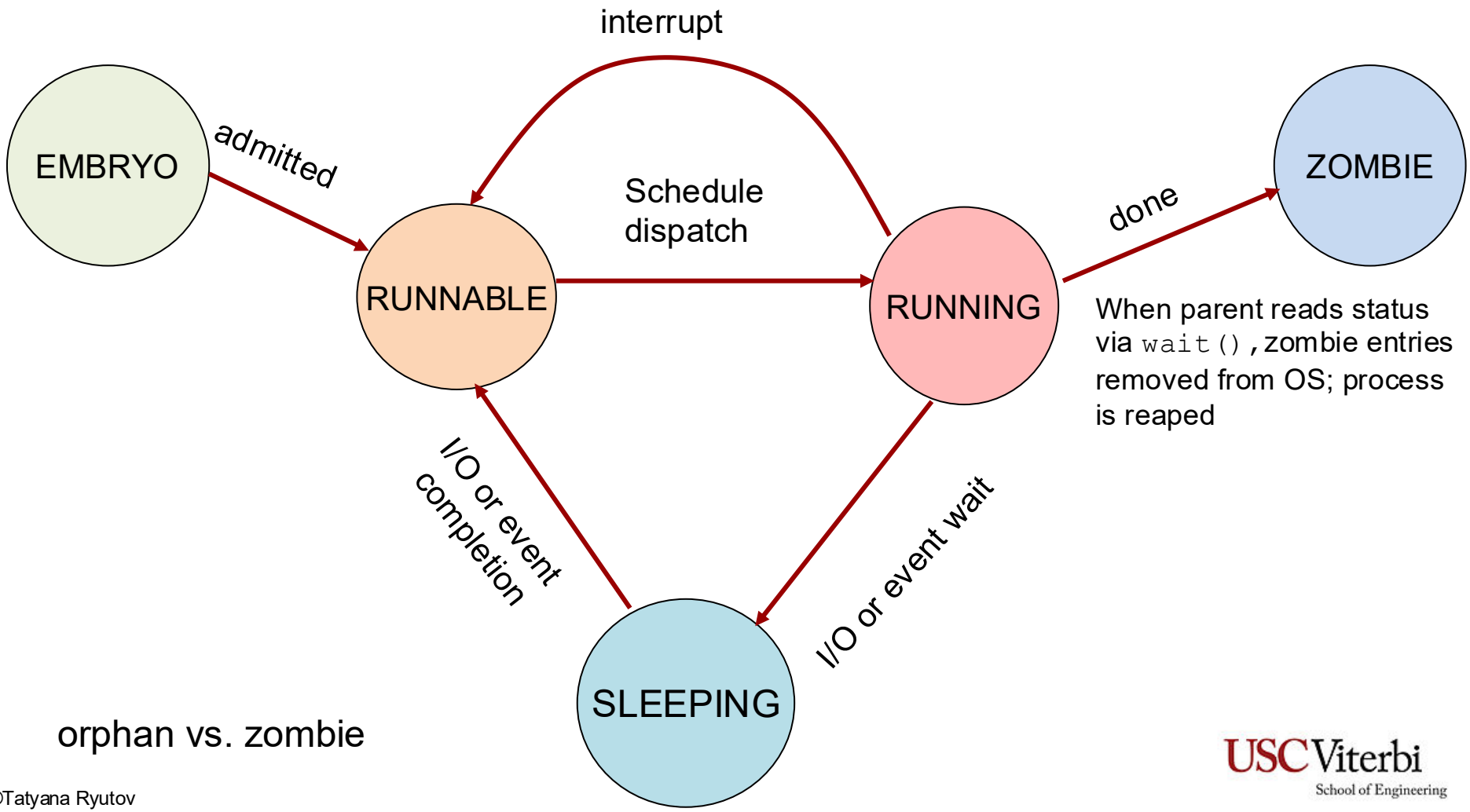
- When a process terminates, it still consumes system resources
  - Table maintained by OS where the child status is stored
  - Called a zombie
- The process of eliminating zombie processes is called reaping
- The simplest method is for parent to call **`wait()`**
  - Parent is given exit status info and kernel deletes zombie child process
- What if parent does not reap?
  - If any parent terminates without reaping a child, then the orphaned child will be reaped by **`init`** process (pid == 1)
  - In long-running processes (e.g., shells, servers) we need explicit reaping

To wait or not to wait: tradeoffs?

©Tatyana Ryutov

Recall: xv6 Process States

```
proc.h:
enum procstate {UNUSED,EMBRYO,SLEEPING,RUNNABLE,RUNNING,ZOMBIE};
```



orphan vs. zombie

©Tatyana Ryutov

USC Viterbi School of Engineering

Example: Zombie

```
void fork7() {
   if (fork() == 0) {
      /* Child */
      printf("Terminating Child, PID = %d\n",
             getpid());
      exit(0);
   } else {
      printf("Running Parent, PID = %d\n",
             getpid());
      while (1); /* Infinite loop */
   }
}
```

forks.c

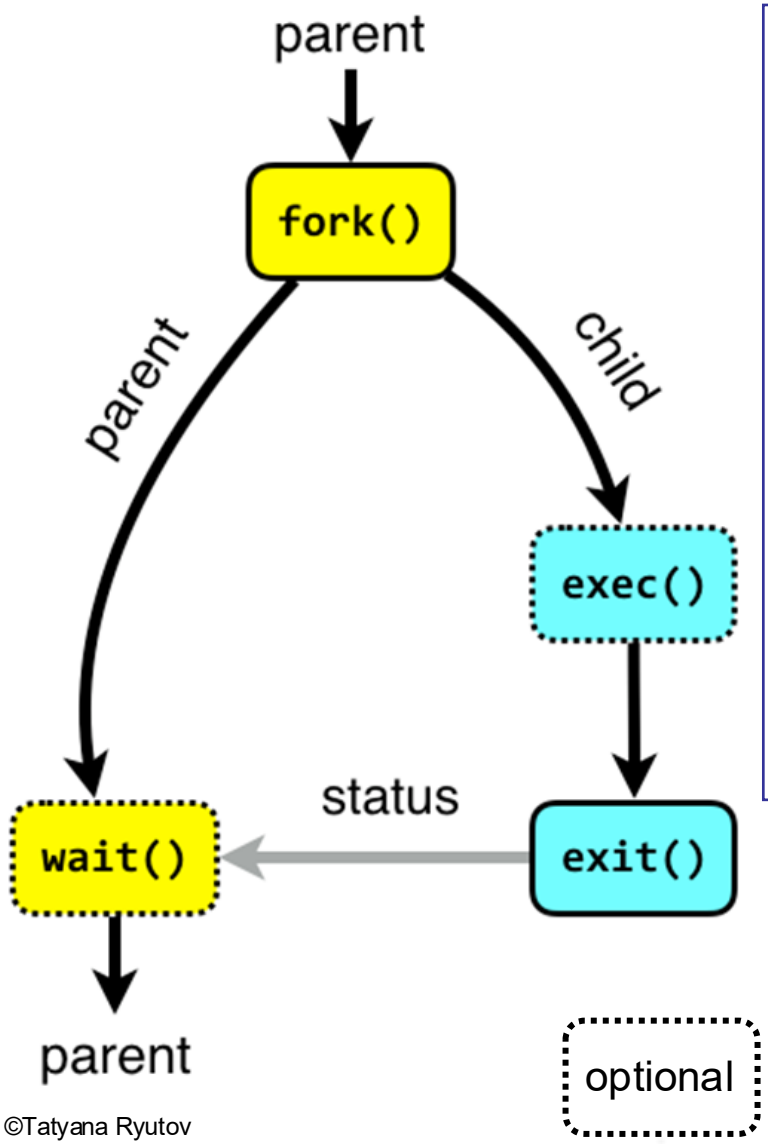
```
linux> ./forks &
[1] 6639
Running Parent, PID = 6639
Terminating Child, PID = 6640
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6639 ttyp9    00:00:03 forks
 6640 ttyp9    00:00:00 forks <defunct>
 6641 ttyp9    00:00:00 ps
linux> kill 6639
[1]    Terminated
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6642 ttyp9    00:00:00 ps
```

`ps` shows child process as "defunct" or "zombie"

killing parent allows child to be reaped by `init`

©Tatyana Ryutov

Unix Process Management Summary




```
int pid = fork();
```
Create a new process that is a clone of its parent

```
exec*("program" [,argvp, envp]);
```
Overlay the calling process virtual memory with a new program, and transfer control to it

```
exit(status);
```
Exit with status, destroying the process: all of the memory and resources associated with it are deallocated. However, the process's entry in the process table remains

```
int pid = wait*(&status);
```
Wait for exit (or other status change) of a child

©Tatyana Ryutov

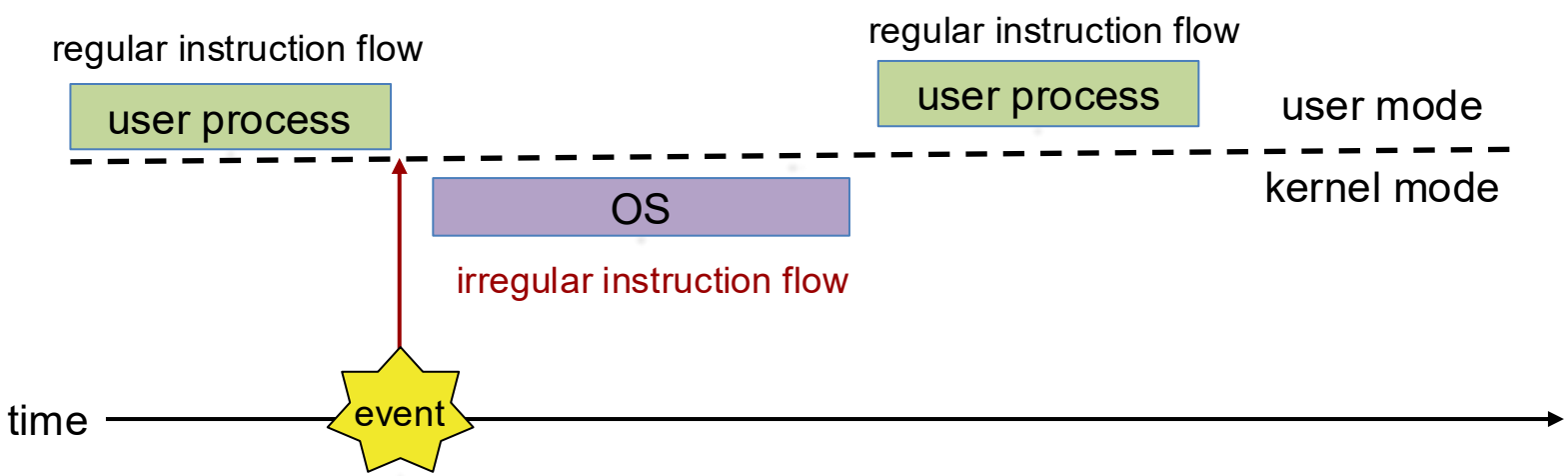
Lecture Outline

- Review
- System calls
- Kernel organization
- Unix process management
- Interrupts

©Tatyana Ryutov

Recall: OS is Event Driven

- Event causes a break in normal execution flow, examples:
  - Error conditions/exceptions
  - HW interrupts raised by devices
  - System calls
- Events automatically cause kernel mode to be entered
- When events occur, automatically call some subroutine (a.k.a. "handler") to handle the issue, then resume normal processing
- How to notify OS about an event?



©Tatyana Ryutov

USC Viterbi School of Engineering

Event Notification

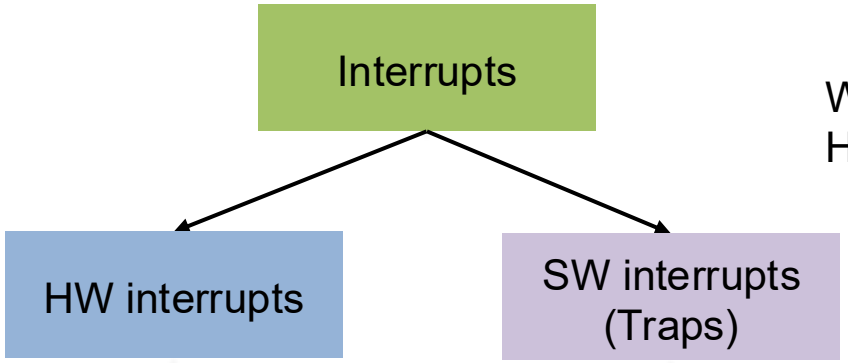
- Two methods:
  1. Polling "busy" loop (responsibility on the processor)
     - CPU periodically checks each device to see if it needs service
  2. Interrupts (responsibility on I/O device)
     - I/O device notifies processor only when it needs attention

Basis for Comparison	Interrupt	Polling
Basic	Device notify CPU that it needs CPU attention	CPU constantly checks device status whether it needs CPU's attention
Mechanism	An interrupt is a hardware mechanism	Polling is a Protocol
Indication	Interrupt-request line indicates that device needs servicing	Command-ready bit indicates the device needs servicing
CPU	CPU is disturbed only when a device needs servicing, which saves CPU cycles	CPU has to wait and check whether a device needs servicing which wastes lots of CPU cycles
Occurrence	An interrupt can occur at any time	CPU polls the devices at regular interval
Efficiency	Interrupt becomes inefficient when devices keep on interrupting the CPU repeatedly	Polling becomes inefficient when CPU rarely finds a device ready for service

©Tatyana Ryutov

Interrupts

- Interrupt is an event that diverts the current execution of a program to a predefined location in the kernel in order to respond to an event
- Software Interrupts (or Traps) are synchronous generated by:
  - SW with a special assembly instruction, e.g., system call
  - CPU, e.g., math error
  - Memory Management Unit, e.g., page fault
- Hardware Interrupts are asynchronous, spawned by event external to CPU
  - Generated by devices when request is complete or by event that requires CPU attention
  - Do not happen at predictable places in the user code

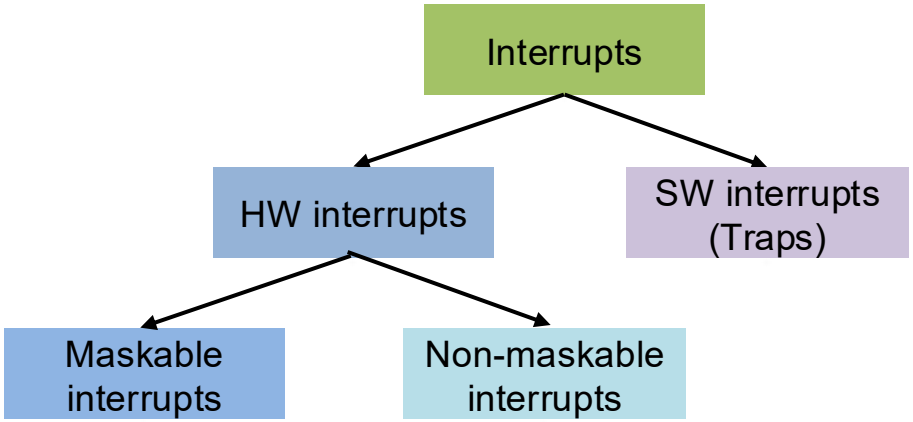


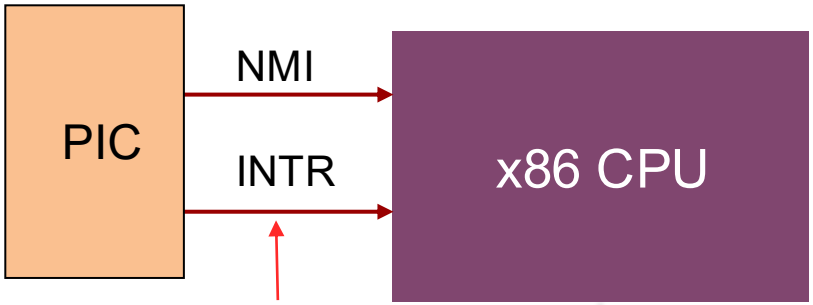
What are the most most common HW interrupts?

©Tatyana Ryutov

Types of HW Interrupts

- HW interrupt is caused by any peripheral device by sending a signal through a specified pin to the processor
- Maskable
  - An interrupt that can be disabled by the CPU instructions
  - For example, interrupts will not be delivered on the INTR line
  - Useful when CPU needs to do something more important
- Non-maskable
  - An interrupt that cannot be disabled by the CPU instructions
  - For example, invoked by NMI line from PIC (Programmable Interrupt Controller)
  - Never ignored, e.g., power failure, non-recoverable hardware errors



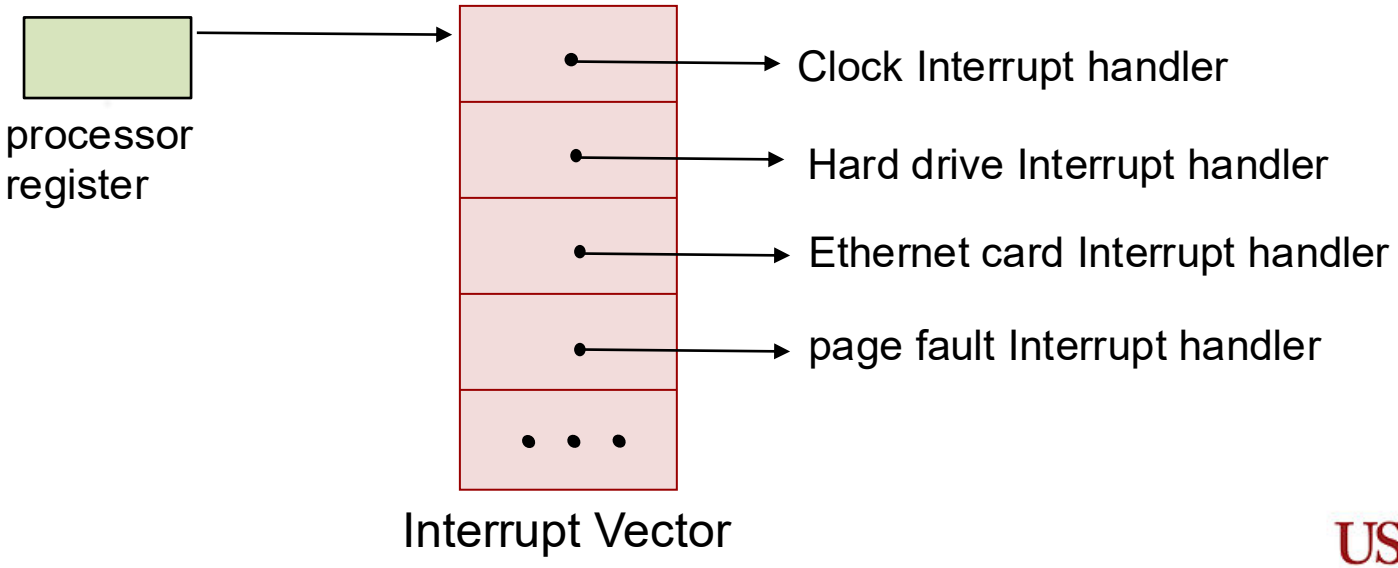


processor recognizes interrupt only if interrupts are enabled

©Tatyana Ryutov

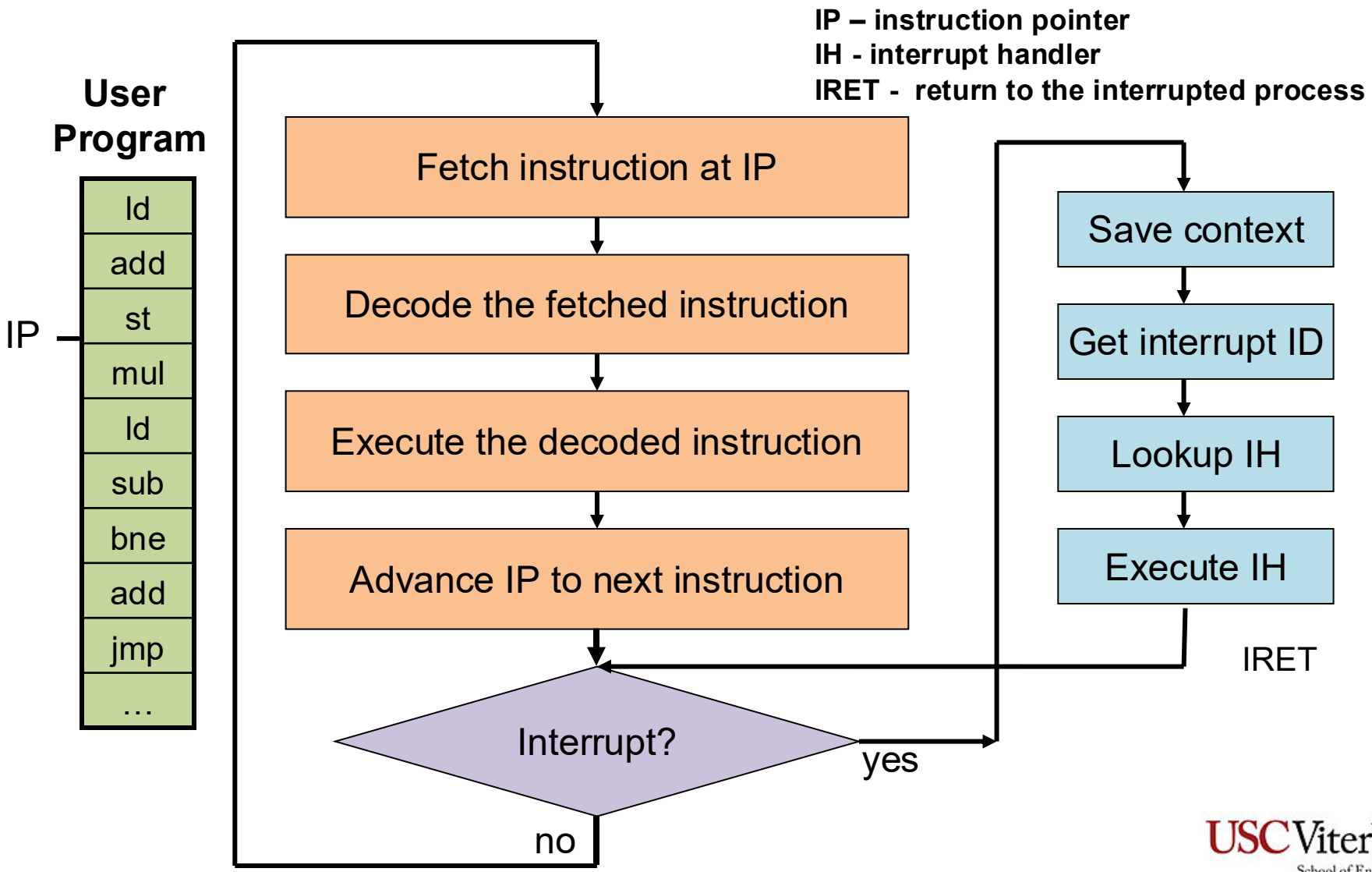
How does the processor know what code to run?

- For protection, execute code in kernel mode only at specific well-defined location (entry point)
- Each interrupt is identified by a number indicating its type
  - E.g., in x86, 14 is a page fault, 3 is a debug breakpoint
- This number is the index into a processor-specific Interrupt Descriptor Table (a.k.a. Interrupt Vector)
  - The table holds start address of the interrupt handlers or Interrupt Service Routines (ISR)
- Interrupt vector is stored in kernel memory, why?



©Tatyana Ryutov

CPU Fetch-Execute Cycle

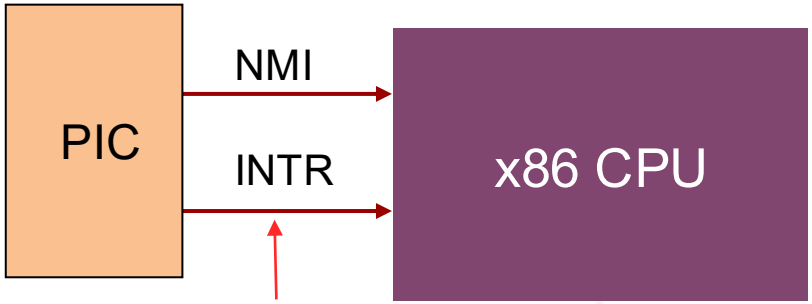


©Tatyana Ryutov

USC Viterbi School of Engineering

Disabling Interrupts

- Interrupt handler runs with interrupts off
  - Re-enabled when interrupt completes
  - Interrupt buffering
- **`IF`** (Interrupt Flag) - system flag bit in the x86 `FLAGS` register
  - Determines whether or not the CPU will handle maskable interrupts
- Kernel can turn interrupts off
  - E.g., when determining the next process/thread to run (scheduling decision)
  - On Intel x86
    - **`CLI`**: disable interrupts (only deferred)
    - **`STI`**: enable interrupts
    - Only applies to the current CPU (on a multicore)



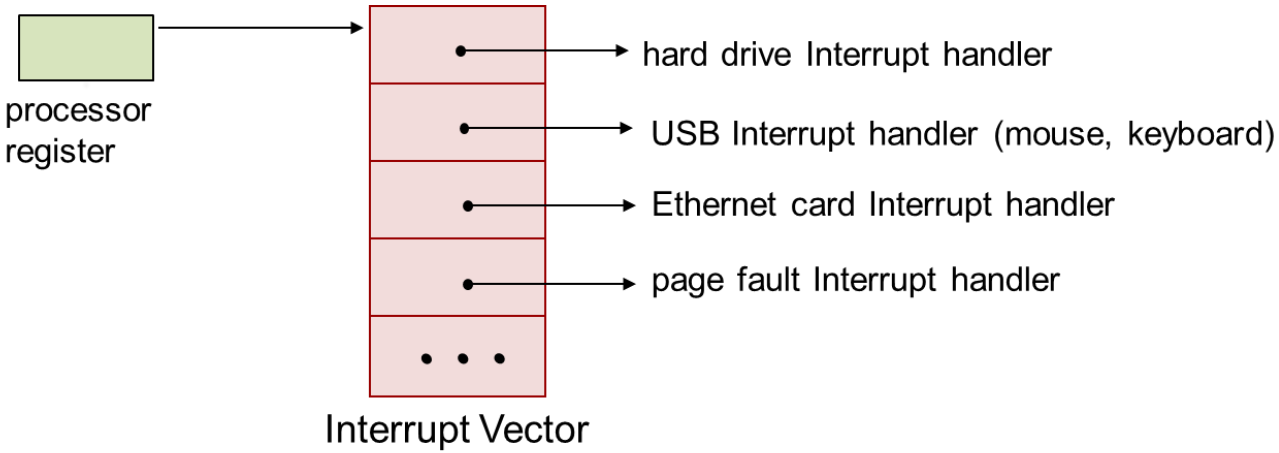
processor recognizes interrupt
only if interrupt flag **`IF`** is set

©Tatyana Ryutov

How do we take interrupts safely?

- Interrupt vector - limited number of entry points into kernel
- Atomic transfer of control from user to kernel mode
  - Single instruction to change:
    - Program counter
    - Stack pointer
    - Memory protection
    - Kernel/user mode

    Executed as one non-divisible operation "all or nothing"
- Support transparent restartable execution



©Tatyana Ryutov

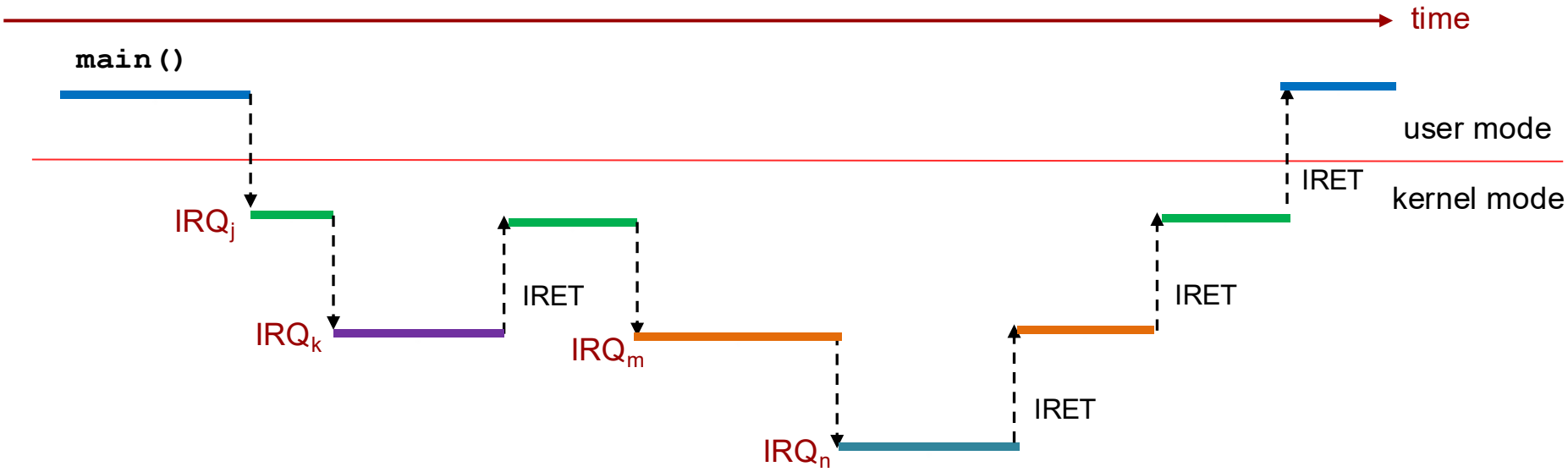
Example: x86 Interrupt

- HW performs the following steps:
  1. Mask interrupts while the processor is in the middle of switching from user mode to kernel mode
  2. Save current stack pointer (SP), program counter (PC), processor status/execution flags (PS) in internal temporary registers
  3. Switch to kernel stack as specified in a special HW register
  4. Push SP, PC, PS on kernel stack
  5. Invoke interrupt handler software that will:
     - Vector through interrupt table (location in a special HW register)
       - Interrupt number serves as index, and the PC is set to the value at this index
       - This starts the handler software that will save registers it might change including the current stack pointer

©Tatyana Ryutov

Can interrupts be preempted?

- Depends on both the processor architecture and the kernel architecture
- On some processors, it is possible for an interrupt to be interrupted by another higher-priority interrupt
  - Example: ARM processors have two interrupt levels: IRQ (normal interrupts) and FIQ (fast interrupts); FIQ can be triggered in the middle of handling an IRQ
  - IRQ is short for interrupt request
- Linux supports nested interrupts: one interrupt can "preempt" another interrupt



Context switches without mode switches

©Tatyana Ryutov

xv6 Interrupts

- 0 to 31 reserved by Intel for different types of processor exceptions
- 32 to 47 used for device interrupts
  - Most can be configured
- 64 (0x40) used for system call interrupt
  - Linux uses 128 (0x80) for its system call
- xv6 interrupt handler entries: **`vector.S`** (generated by vectors.pl)

```
# generated by vectors.pl - do not edit
# handlers
.globl alltraps
.globl vector0
vector0:
  pushl $0
  pushl $0
  jmp alltraps
.globl vector1
vector1:
  pushl $0
  pushl $1
  jmp alltraps
.globl vector2
vector2:
  pushl $0
  pushl $2
  jmp alltraps
.globl vector3
```

```
# vector table
.data
.globl vectors
vectors:
  .long vector0
  .long vector1
  .long vector2
```

©Tatyana Ryutov

xv6 Trap Instruction (`int n`)

- For HW interrupts, device sends a signal to CPU, and CPU executes **`int`** instruction
  - Trap instruction has a parameter (`int n`), indicating type of interrupt
- The following steps are performed by CPU:
  - Fetch n^{th} entry in IDT (CPU knows memory address of IDT)
  - Save stack pointer (esp) to internal register
  - Switch esp to kernel stack of process (CPU knows location of kernel stack of current process)
  - On kernel stack, save old esp, eip (where execution stopped before interrupt occurred, so that it can be resumed later)
  - Load new eip from IDT, points to kernel trap handler
- Result: ready to run kernel trap handler code, on kernel stack of the process

©Tatyana Ryutov

Trap Frame on the Kernel Stack

- Trap frame: state is pushed on kernel stack during trap handling
- **`int n`** instruction has so far only pushed the bottom few entries of trap frame

```
struct trapframe {
  // registers as pushed by pusha
  uint edi;
  uint esi;
  uint ebp;
  uint ebx;
  uint edx;
  uint ecx;
  uint eax;

  // rest of trap frame
  ushort gs;
  ushort padding1;
  ushort fs;
  ushort padding2;
  ushort es;
  ushort padding3;
  ushort ds;
  ushort padding4;
  uint trapno;

  // below here defined by x86 hardware
  uint err;
  uint eip;
  ushort cs;
  ushort padding5;
  uint eflags;

  // below here only when crossing rings, such as from user to kernel
  uint esp;
  ushort ss;
  ushort padding6;
};
```

©Tatyana Ryutov

Kernel Trap Handler (alltraps)

- IDT entries for all interrupts will set eip to point to the kernel trap handler “alltraps”
- alltraps assembly code pushes remaining registers to complete trapframe on kernel stack
- Invokes C trap handling function named “trap”

trapasm.S

```
#include "mmu.h"

  # vectors.S sends all traps here.
.globl alltraps
alltraps:
  # Build trap frame.
  pushl %ds
  pushl %es
  pushl %fs
  pushl %gs
  pushal

  # Set up data segments.
  movw $(SEG_KDATA<<3), %ax
  movw %ax, %ds
  movw %ax, %es

  # Call trap(tf), where tf=%esp
  pushl %esp
  call trap
  addl $4, %esp

  # Return falls through to trapret...
.globl trapret
trapret:
  popal
  popl %gs
  popl %fs
  popl %es
  popl %ds
  addl $0x8, %esp  # trapno and errcode
  iret
```

©Tatyana Ryutov

trap.c Handler Function

- C trap handler performs different actions based on kind of event
  - If system call, “int n” is invoked with “n” equal to a value T_SYSCALL (in usys.S), If interrupt from a device, corresponding device-related code is called
  - On timer interrupt, a process “yields” CPU to scheduler

```c
void
trap(struct trapframe *tf)
{
  if(tf->trapno == T_SYSCALL){
    if(proc->killed)
      exit();
    proc->tf = tf;
    syscall();
    if(proc->killed)
      exit();
    return;
  }
```

trap.c

```c
switch(tf->trapno){
case T_IRQ0 + IRQ_TIMER:
  if(cpunum() == 0){
    acquire(&tickslock);
    ticks++;
    wakeup(&ticks);
    release(&tickslock);
  }
```

trap.c

```c
    lapiceoi();
    break;
  case T_IRQ0 + IRQ_IDE:  // 46
    ideintr();
    lapiceoi();
    break;
  case T_IRQ0 + IRQ_IDE+1:
    // Bochs generates spurious IDE1 interrupts.
    break;
  case T_IRQ0 + IRQ_KBD:
    kbdintr();
    lapiceoi();
    break;
  case T_IRQ0 + IRQ_COM1:
    uartintr();
    lapiceoi();
    break;
```

trap.c

```c
// Force process to give up CPU on clock tick.
// If interrupts were on while locks held, would need to check nlock.
if(proc && proc->state == RUNNING && tf->trapno == T_IRQ0+IRQ_TIMER)
  yield();
```

trap.c

©Tatyana Ryutov

Return from Event

- Return from trap instruction **`iret`** does the opposite of **`int`**
  - Pop values pushed by **`int`**
  - Change privilege level
- Execution of pre-trap code can resume

trapasm.S

```
#include "mmu.h"

  # vectors.S sends all traps here.
.globl alltraps
alltraps:
  # Build trap frame.
  pushl %ds
  pushl %es
  pushl %fs
  pushl %gs
  pushal

  # Set up data segments.
  movw $(SEG_KDATA<<3), %ax
  movw %ax, %ds
  movw %ax, %es

  # Call trap(tf), where tf=%esp
  pushl %esp
  call trap
  addl $4, %esp

  # Return falls through to trapret...
.globl trapret
trapret:
  popal
  popl %gs
  popl %fs
  popl %es
  popl %ds
  addl $0x8, %esp  # trapno and errcode
  iret
```

©Tatyana Ryutov

Summary of xv6 Even Handling

- Events (software or hardware interrupts) cause CPU to run **`int n`** instruction and “trap” to OS
- The trap instruction causes CPU to switch esp to kernel stack, eip to kernel trap handling code
- Pre-trap CPU state is saved on kernel stack in the trap frame (by **`int`** instruction + alltraps code)
- Kernel trap handler handles trap and returns from trap to whatever was running before the trap

©Tatyana Ryutov